When Clustering Metrics Disagree

A contingency-matrix study of purity, maximum-matching accuracy, precision, recall and F-measure under fragmentation, merging and class imbalance

> **Research question.** How can evaluation metrics produce conflicting conclusions about clustering quality?

Name _____

Roll / Student ID _____

Course _____

Instructor _____

Date of submission _____

Contents

Abstract

Introduction and method

1. Metrics, all derived from the contingency matrix
   Worked contingency matrix (imbalanced data, k-means with k = 8)
2. Experiment 1: varying the number of clusters
3. Experiment 2: fragmentation vs merging
   One split vs one merge
4. Experiment 3: class imbalance
5. Experiment 4: do metrics rank the same solutions the same way?
   Full pool (16 solutions)
   Algorithmic solutions only (11 solutions)
   Concrete reversals (imbalanced data, algorithmic solutions)
   Full scores (imbalanced data)
6. Answer: how metrics come to conflicting conclusions

References

Appendix A. Reproducing the results

Appendix B. Source code

Abstract

External clustering metrics compress the contingency matrix between predicted clusters and true classes into a single number, and each compression encodes a different notion of error. We implement purity, inverse purity, maximum-matching accuracy, pair-counting precision, recall and F1, the class F-measure, BCubed, ARI and NMI from one contingency matrix. We then evaluate them under controlled fragmentation and merging, a class-imbalance sweep, k-means with k = 1…15, and a ranking of 16 candidate solutions on balanced and imbalanced data.

One-sided metrics are maximised by degenerate clusterings. On imbalanced data, purity peaks at k = 13 and pair recall at k = 1. Metrics without chance correction prefer a single all-inclusive cluster over a near-perfect solution once the majority class exceeds a metric-specific share of the data: $p \geq 0.70$ for accuracy and $p \geq 0.80$ for BCubed F1. ARI and NMI never do. Among 11 solutions produced by clustering algorithms, the mean Kendall rank correlation between six two-sided metrics drops from 0.95 on balanced data to 0.83 on imbalanced data.

Conflicts arise from four design choices hidden inside each metric: which error type it penalises, whether it counts points or pairs, how it matches clusters to classes, and whether it corrects for chance. Class imbalance and a wrong number of clusters amplify all four.

Introduction and method

Clustering is unsupervised, so when ground-truth labels exist the natural evaluation is to compare the predicted partition with the true one. Cluster identifiers are arbitrary, however, so every external metric first builds the **contingency matrix** (true classes × predicted clusters) and then summarises it. The research question is how the choice of summary can change the verdict.

Two failure modes are central:

- **Fragmentation:** a single true class is split across several clusters.
- **Merging:** several true classes share one cluster.

Class imbalance matters because it changes how much each class contributes to a point-weighted or pair-weighted score.

Data. The data are two-dimensional Gaussian blobs with 5 classes around fixed centres (standard deviation 1.3, so neighbouring classes overlap). The balanced version has 5 × 240 points; the imbalanced one has class sizes [700, 250, 120, 80, 50]. The controlled experiments use synthetic label vectors so that the type and size of every error is known exactly.

Clusterings. k-means (k = 1…15), agglomerative clustering (ward, average and single linkage), a Gaussian mixture model, and constructed corruptions of the truth (fragmenting a class, merging classes, random label noise, one cluster, all singletons).

Comparison of rankings. For each metric, the solutions are ranked from best to worst. Agreement between two metrics is measured by Kendall's tau over solution scores and by the *disagreement rate*: the fraction of solution pairs, among those not tied under either metric, that the two metrics order in opposite directions.

The implementation is verified by unit tests against textbook values and scikit-learn (Appendix A). Every table and figure is regenerated by `python experiments/run_all.py`.

1. Metrics, all derived from the contingency matrix

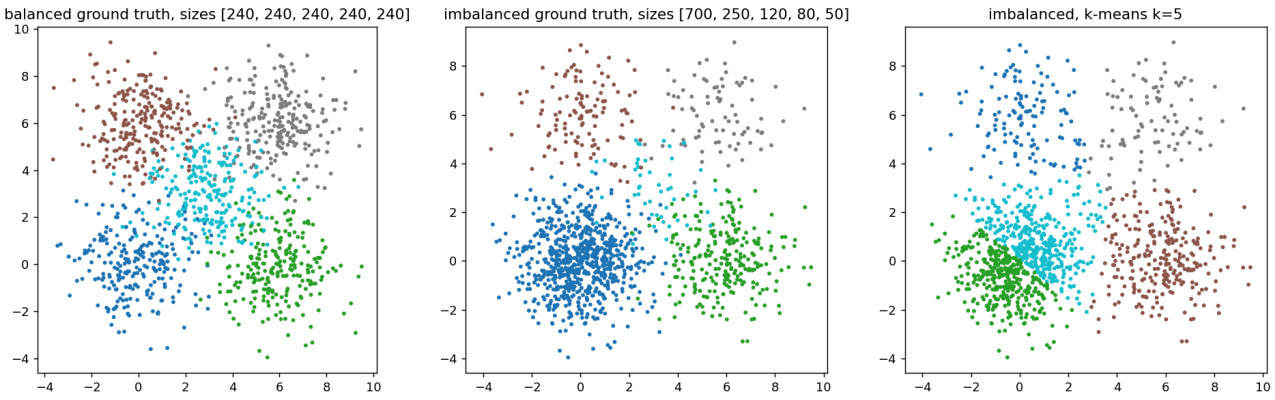
Let `n_ij` be the number of points of true class *i* in predicted cluster *j*, `a_i` the class sizes, `b_j` the cluster sizes and `N` the number of points.

Metric	Definition	Error it ignores
Purity	Σ_j max_i n_ij / N	fragmentation (all singletons = 1)
Inverse purity	Σ_i max_j n_ij / N	merging (one cluster = 1)
Max-matching accuracy	best one-to-one class↔cluster assignment (Hungarian), matched points / N	none, but surplus clusters count fully as errors
Pair precision	pairs together in both / pairs together in prediction	fragmentation
Pair recall	pairs together in both / pairs together in truth	merging
Pair F1	harmonic mean of pair precision and recall	weights classes by size²
Class F-measure	Σ_i (a_i/N) max_j F(i, j) (Larsen & Aone)	one cluster may serve several classes
BCubed F1	per-point precision / recall, then F1	—
ARI	Rand index corrected for chance	—
NMI	mutual information / mean entropy	mild bias towards more clusters

Worked contingency matrix (imbalanced data, k-means with k = 8)

	cluster 0	cluster 1	cluster 2	cluster 3	cluster 4	cluster 5	cluster 6	cluster 7	class size
class 0	0	194	251	0	237	1	8	9	700
class 1	130	0	2	0	0	0	99	19	250
class 2	0	1	0	1	0	110	0	8	120
class 3	0	0	0	75	0	0	0	5	80
class 4	1	2	3	1	0	3	0	40	50
cluster size	131	197	256	77	237	114	107	81	1200
majority class	1	0	0	3	0	2	1	4	

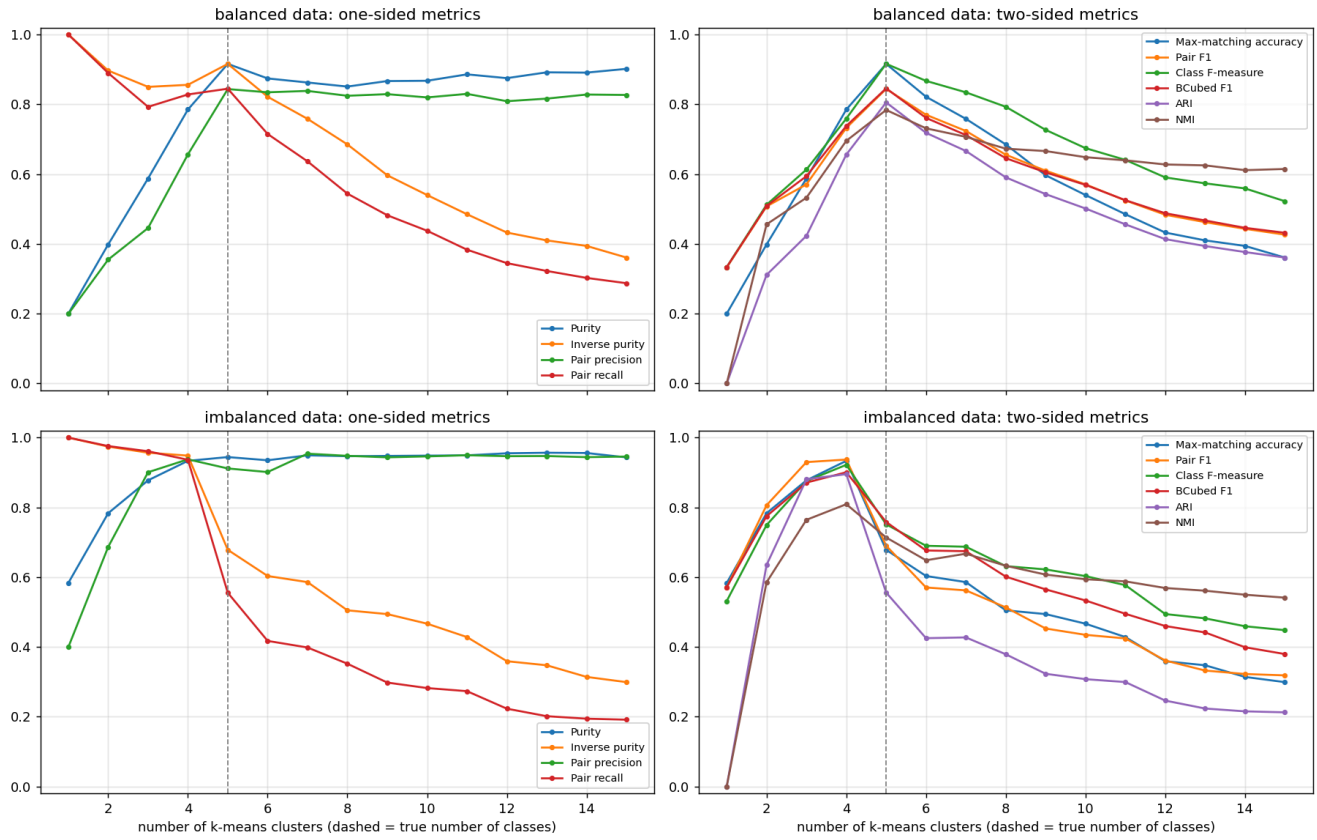
Purity credits each cluster with its majority class, so it is 0.947. Max-matching accuracy may use each class only once, so the three surplus clusters count as errors and accuracy drops to 0.505. The Hungarian mapping (cluster → class) is `{2: 0, 0: 1, 5: 2, 3: 3, 7: 4}`.



2. Experiment 1: varying the number of clusters

K-means with k = 1…15 on 5-class blobs (balanced: 5 × 240 points; imbalanced: [700, 250, 120, 80, 50]).

Metric	best k (balanced)	best k (imbalanced)
Purity	5	13
Inverse purity	1	1
Pair precision	5	7
Pair recall	1	1
Max-matching accuracy	5	4
Pair F1	5	4
Class F-measure	5	4
BCubed F1	5	4
ARI	5	4
NMI	5	4



The one-sided metrics are pulled towards the extremes. Inverse purity and pair recall are maximised by a single cluster (k = 1). On the imbalanced data, purity peaks at k = 13 and pair precision at k = 7. Both can be inflated simply by asking for more clusters.

On the imbalanced data, every two-sided metric prefers k = 4 to the true k = 5. The contingency matrices show why. At k = 5, k-means spends a centroid splitting the 700-point class in half, and the 50-point central class still gets no cluster of its own:

	cluster 0	cluster 1	cluster 2	cluster 3	cluster 4
class 0	2	353	6	0	339
class 1	0	0	245	1	4
class 2	117	0	0	1	2

	cluster 0	cluster 1	cluster 2	cluster 3	cluster 4
class 3	1	0	0	79	0
class 4	11	0	10	9	20

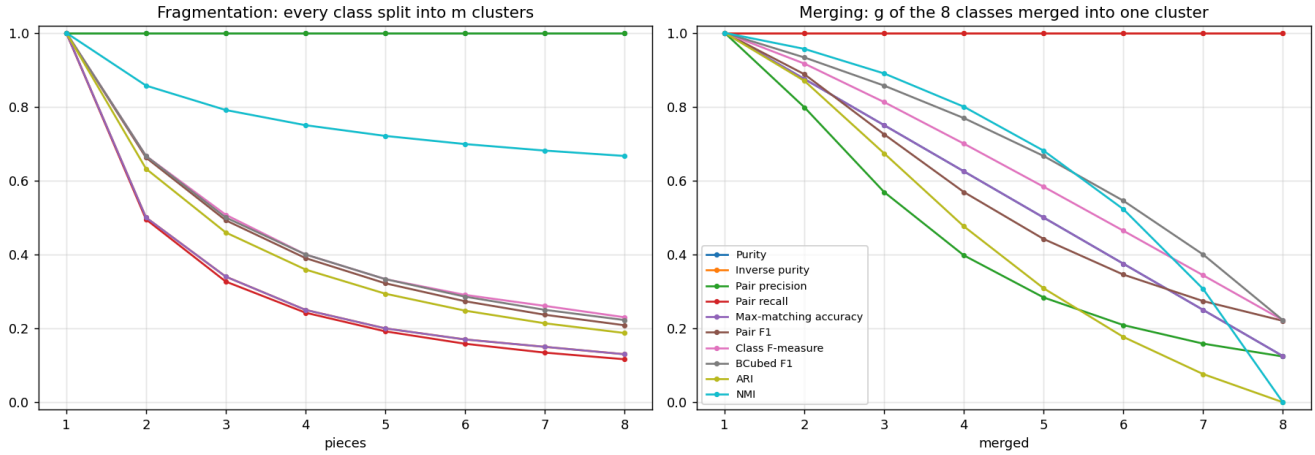
At k = 4, the majority class stays intact and only the tiny class is lost:

	cluster 0	cluster 1	cluster 2	cluster 3
class 0	12	676	12	0
class 1	0	2	247	1
class 2	119	0	0	1
class 3	2	0	0	78
class 4	17	6	18	9

Purity barely notices the split (0.944 at k = 5 vs 0.933 at k = 4). Max-matching accuracy collapses from 0.933 to 0.678 because one half of the majority class can no longer be matched. This is the same data and the same pair of clusterings, but purity and accuracy reach opposite conclusions.

3. Experiment 2: fragmentation vs merging

Ground truth of 8 classes × 100 points; either split every class into *m* clusters or merge *g* classes into one.



pieces	Purity	Inverse purity	Pair precision	Pair recall	Max-matching accuracy	Pair F1	Class F-measure	BCubed F1	ARI	NMI
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.000	0.500	1.000	0.495	0.500	0.662	0.667	0.667	0.632	0.857
3	1.000	0.340	1.000	0.327	0.340	0.492	0.507	0.500	0.459	0.791
4	1.000	0.250	1.000	0.242	0.250	0.390	0.400	0.400	0.359	0.750
5	1.000	0.200	1.000	0.192	0.200	0.322	0.333	0.333	0.294	0.721
6	1.000	0.170	1.000	0.158	0.170	0.273	0.291	0.286	0.248	0.699
7	1.000	0.150	1.000	0.134	0.150	0.237	0.261	0.250	0.214	0.681
8	1.000	0.130	1.000	0.116	0.130	0.208	0.230	0.223	0.187	0.667

Fragmentation (rows = pieces per class).

merged	Purity	Inverse purity	Pair precision	Pair recall	Max-matching accuracy	Pair F1	Class F-measure	BCubed F1	ARI	NMI
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	0.875	1.000	0.798	1.000	0.875	0.888	0.917	0.933	0.870	0.957
3	0.750	1.000	0.569	1.000	0.750	0.725	0.812	0.857	0.674	0.890
4	0.625	1.000	0.398	1.000	0.625	0.569	0.700	0.769	0.476	0.800
5	0.500	1.000	0.284	1.000	0.500	0.442	0.583	0.667	0.308	0.681
6	0.375	1.000	0.209	1.000	0.375	0.346	0.464	0.545	0.177	0.523
7	0.250	1.000	0.159	1.000	0.250	0.274	0.344	0.400	0.076	0.307
8	0.125	1.000	0.124	1.000	0.125	0.220	0.222	0.222	0.000	0.000

Merging (rows = number of classes merged).

Fragmentation is invisible to purity, pair precision and BCubed precision, all of which stay at 1.0. Merging is invisible to inverse purity, pair recall and BCubed recall. So any single one-sided score can rank a badly fragmented solution above a mildly merged one, or the reverse.

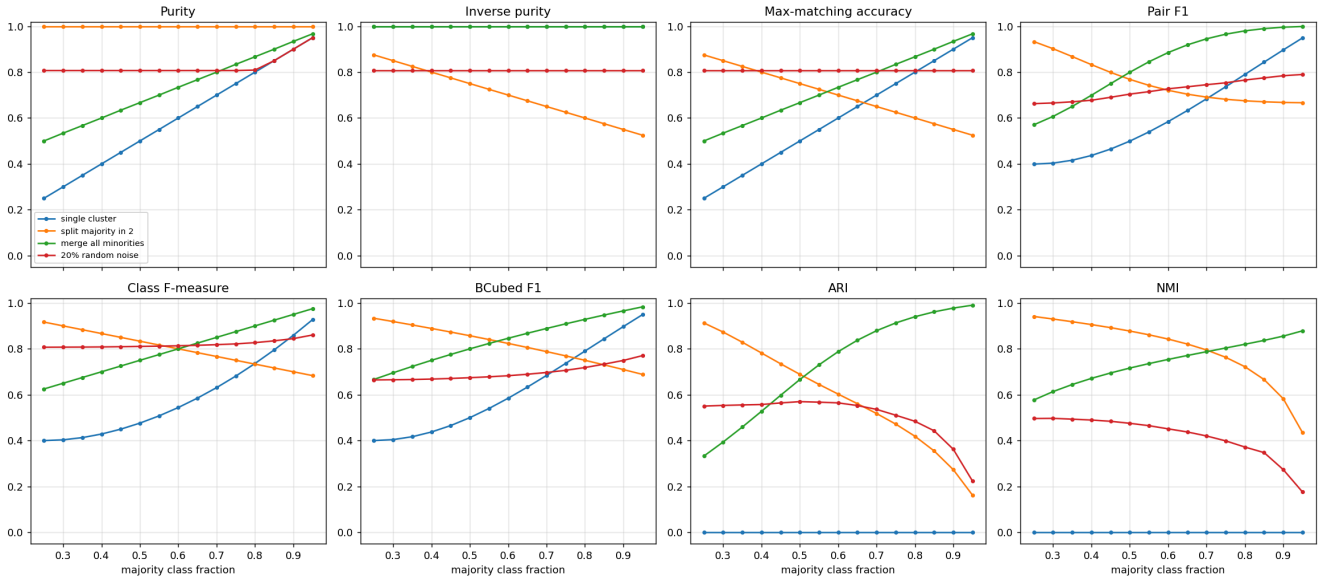
One split vs one merge

Scenario	Purity	Inverse purity	Pair precision	Pair recall	Max-matching accuracy	Pair F1	Class F-measure	BCubed F1	ARI	NMI
balanced: split one class in 2	1.000	0.875	1.000	0.874	0.875	0.933	0.917	0.933	0.913	0.941
balanced: merge two classes	0.750	1.000	0.666	1.000	0.750	0.799	0.833	0.857	0.714	0.857
imbalanced: split majority in 2	1.000	0.650	1.000	0.532	0.650	0.695	0.767	0.788	0.520	0.790
imbalanced: merge two smallest	0.950	1.000	0.981	1.000	0.950	0.991	0.955	0.966	0.980	0.945
imbalanced: merge majority + smallest	0.950	1.000	0.882	1.000	0.950	0.937	0.932	0.951	0.859	0.888

The same structural mistake costs very different amounts depending on which class it hits. On imbalanced data, splitting the majority class in half costs pair F1 a lot because it destroys a quadratic number of pairs. NMI, by contrast, stays relatively high (0.790 against an ARI of 0.520). Merging the two smallest classes barely registers under pair-based metrics (pair F1 0.991), but max-matching accuracy charges for every point of the unmatched class (0.950).

4. Experiment 3: class imbalance

Four classes and 1000 points; the majority class holds a fraction p, and the rest is split evenly over three minority classes.



When does a degenerate one-cluster answer beat a near-perfect answer that only splits the majority class in two?

Metric	single cluster ≥ split-majority when
Purity	never
Inverse purity	always
Max-matching accuracy	p ≥ 0.70
Pair F1	p ≥ 0.75
Class F-measure	p ≥ 0.80
BCubed F1	p ≥ 0.80
ARI	never
NMI	never

Best solution per metric as imbalance grows:

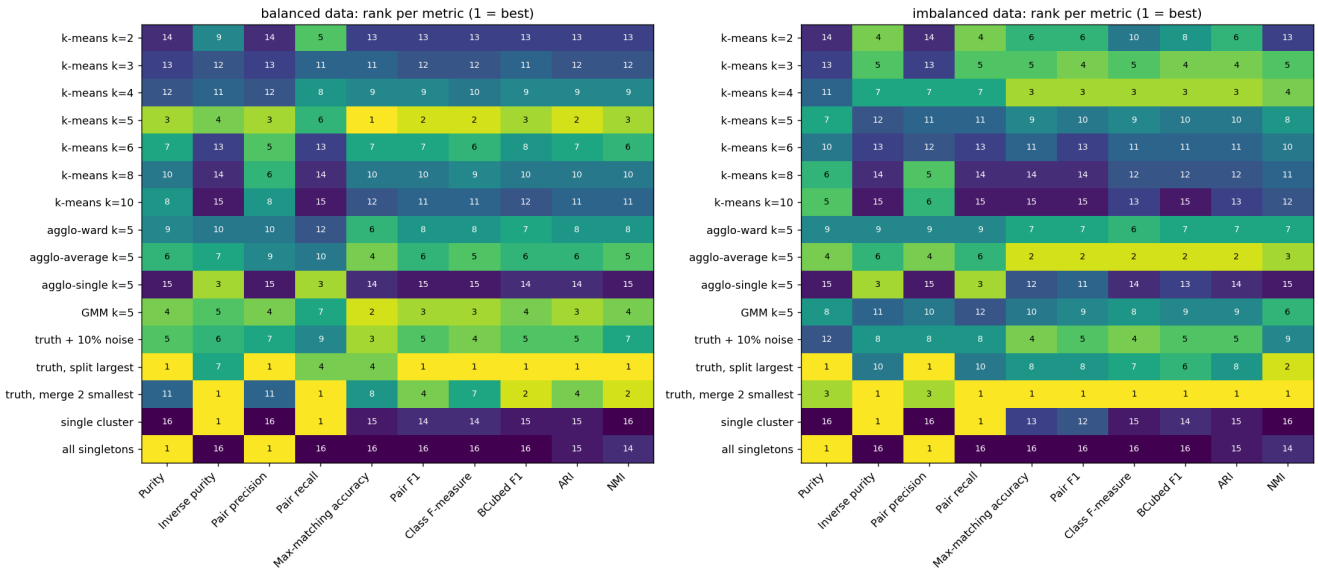
Metric	p = 0.25	p = 0.50	p = 0.75	p = 0.95
Purity	split majority in 2	split majority in 2	split majority in 2	split majority in 2
Inverse purity	single cluster / merge all minorities	single cluster / merge all minorities	single cluster / merge all minorities	single cluster / merge all minorities
Max-matching accuracy	split majority in 2	20% random noise	merge all minorities	merge all minorities
Pair F1	split majority in 2	merge all minorities	merge all minorities	merge all minorities
Class F-measure	split majority in 2	split majority in 2	merge all minorities	merge all minorities
BCubed F1	split majority in 2	split majority in 2	merge all minorities	merge all minorities
ARI	split majority in 2	split majority in 2	merge all minorities	merge all minorities
NMI	split majority in 2	split majority in 2	merge all minorities	merge all minorities

At p = 0.95 the single cluster scores purity 0.950, accuracy 0.950 and pair F1 0.949, but ARI 0.000 and NMI 0.000. Metrics without chance correction inherit the majority-class baseline, so their absolute values become uninterpretable under imbalance. Their rankings can also flip in favour of trivial solutions.

5. Experiment 4: do metrics rank the same solutions the same way?

Each dataset gets 16 candidate solutions. 11 are produced by algorithms: k-means for several k, agglomerative clustering (ward, average and single linkage) and a GMM. The other 5 are controlled corruptions of the truth (10% noise, split largest class, merge two smallest classes) plus the two degenerate answers (one cluster, all singletons). Rankings are analysed for the full pool and for the 11 algorithmic solutions alone, which is the realistic model-selection setting.

Full pool (16 solutions)



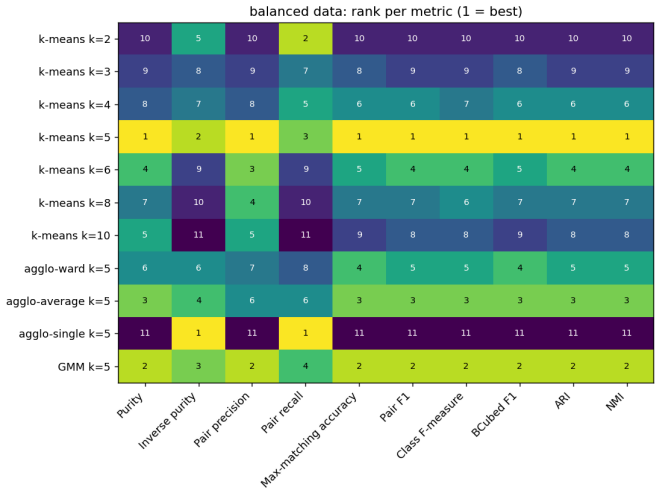
One controlled corruption, *truth, merge 2 smallest*, is closer to the truth than any algorithm output, and every two-sided metric ranks it first. The other corruptions do not dominate: *truth + 10% noise*, for example, loses to the best algorithms on every two-sided metric.

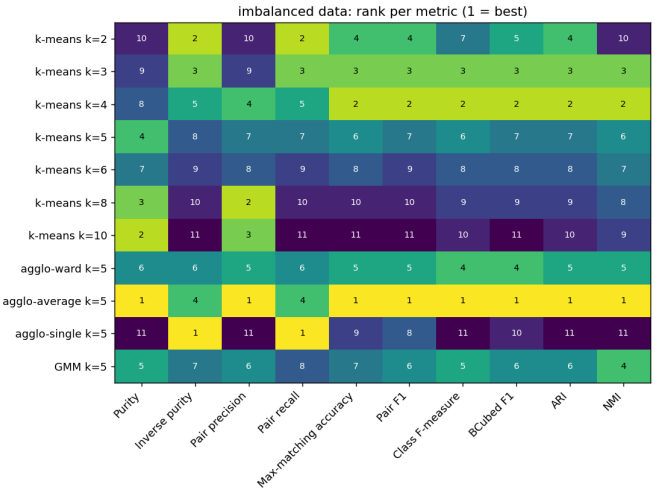
Metric	Top-ranked solution(s)
Purity	truth, split largest, all singletons
Inverse purity	truth, merge 2 smallest, single cluster
Pair precision	truth, split largest, all singletons
Pair recall	truth, merge 2 smallest, single cluster
Max-matching accuracy	truth, merge 2 smallest
Pair F1	truth, merge 2 smallest
Class F-measure	truth, merge 2 smallest
BCubed F1	truth, merge 2 smallest
ARI	truth, merge 2 smallest
NMI	truth, merge 2 smallest

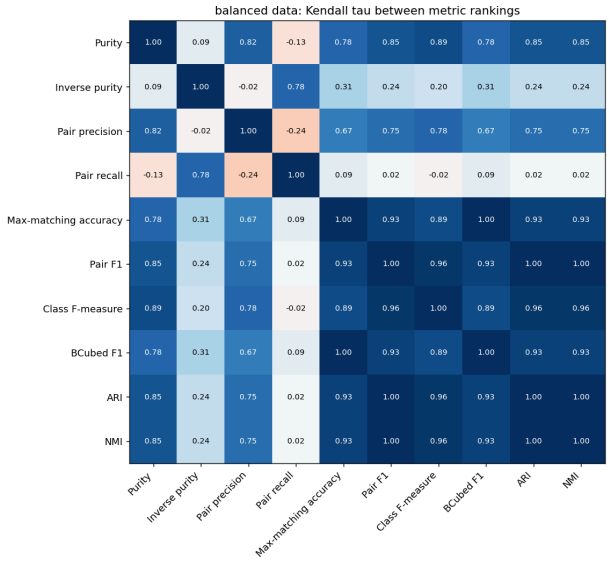
(imbalanced data; ties share rank 1)

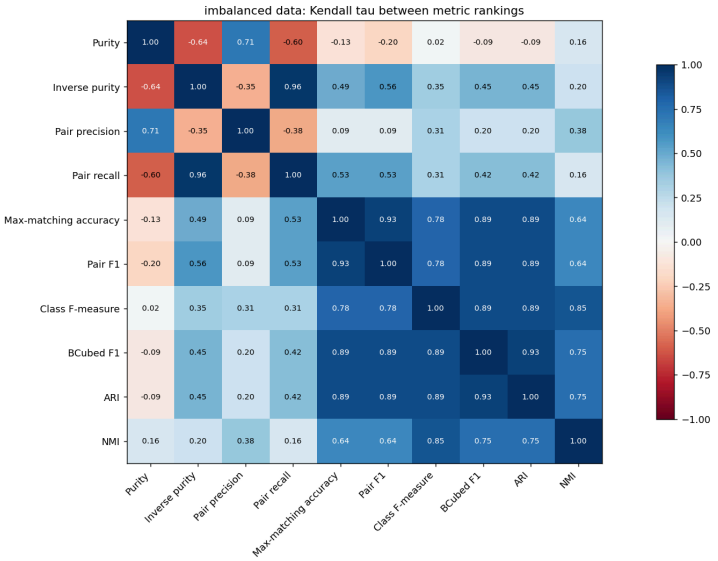
Even here, the one-sided metrics crown a degenerate answer. Purity and pair precision give their maximum of 1.0 to "all singletons" as well as to the fragmented truth, while inverse purity and pair recall give theirs to "single cluster" as well as to the merged truth.

Algorithmic solutions only (11 solutions)









Statistic	balanced / all	imbalanced / all	balanced / algorithms	imbalanced / algorithms
mean Kendall tau (all 10 metrics)	0.520	0.432	0.593	0.398
min Kendall tau (all 10 metrics)	-0.286	-0.504	-0.236	-0.636
mean disagreement rate (all 10 metrics)	0.239	0.284	0.204	0.301
mean Kendall tau (6 two-sided metrics)	0.877	0.817	0.949	0.825
min Kendall tau (6 two-sided metrics)	0.745	0.600	0.891	0.636
mean disagreement rate (6 two-sided metrics)	0.060	0.091	0.025	0.087
mean rank spread across two-sided metrics	2.000	2.938	0.545	1.818

"Rank spread" is the average, over solutions, of (worst rank − best rank) that the six two-sided metrics give the same solution.

On the imbalanced data, the best algorithm according to the two-sided metrics is *agglo-average k=5* by Max-matching accuracy, Pair F1, Class F-measure, BCubed F1, ARI, NMI. They do not agree on the worst one: *k-means k=10* by Max-matching accuracy, Pair F1, BCubed F1; *agglo-single k=5* by Class F-measure, ARI, NMI. The disagreements are not limited to near-ties either. The widest gap among the two-sided metrics is for *k-means k=2*, which is ranked 4 of 11 by Max-matching accuracy but 10 by NMI. Including one-sided metrics widens it further: over-split k-means with k = 10 is ranked 2 by purity and 11 by accuracy.

Concrete reversals (imbalanced data, algorithmic solutions)

- **Max-matching accuracy vs NMI** reverse the order of 10 solution pairs, e.g.
- Max-matching accuracy prefers *agglo-single k=5* (0.585 vs 0.467); NMI prefers *k-means k=10* (0.594 vs 0.012)
- Max-matching accuracy prefers *k-means k=2* (0.783 vs 0.676); NMI prefers *GMM k=5* (0.720 vs 0.585)
- Max-matching accuracy prefers *k-means k=2* (0.783 vs 0.678); NMI prefers *k-means k=5* (0.714 vs 0.585)
- **Pair F1 vs ARI** reverse the order of 3 solution pairs, e.g.
- Pair F1 prefers *agglo-single k=5* (0.572 vs 0.435); ARI prefers *k-means k=10* (0.308 vs 0.006)
- Pair F1 prefers *agglo-single k=5* (0.572 vs 0.513); ARI prefers *k-means k=8* (0.379 vs 0.006)
- Pair F1 prefers *agglo-single k=5* (0.572 vs 0.570); ARI prefers *k-means k=6* (0.425 vs 0.006)
- **BCubed F1 vs Max-matching accuracy** reverse the order of 3 solution pairs, e.g.
- BCubed F1 prefers *k-means k=8* (0.602 vs 0.574); Max-matching accuracy prefers *agglo-single k=5* (0.585 vs 0.505)
- BCubed F1 prefers *GMM k=5* (0.762 vs 0.758); Max-matching accuracy prefers *k-means k=5* (0.678 vs 0.676)
- BCubed F1 prefers *agglo-ward k=5* (0.788 vs 0.775); Max-matching accuracy prefers *k-means k=2* (0.783 vs 0.781)
- **Class F-measure vs ARI** reverse the order of 3 solution pairs, e.g.
- Class F-measure prefers *agglo-ward k=5* (0.833 vs 0.749); ARI prefers *k-means k=2* (0.635 vs 0.624)
- Class F-measure prefers *GMM k=5* (0.754 vs 0.749); ARI prefers *k-means k=2* (0.635 vs 0.560)
- Class F-measure prefers *k-means k=5* (0.751 vs 0.749); ARI prefers *k-means k=2* (0.635 vs 0.556)

Full scores (imbalanced data)

Solution	Purity	Inverse purity	Pair precision	Pair recall	Max-matching accuracy	Pair F1	Class F-measure	BCubed F1	ARI	NMI
k-means k=2	0.783	0.974	0.687	0.976	0.783	0.806	0.749	0.775	0.635	0.585
k-means k=3	0.877	0.957	0.901	0.961	0.877	0.930	0.875	0.871	0.881	0.764
k-means k=4	0.933	0.948	0.938	0.936	0.933	0.937	0.922	0.901	0.895	0.809
k-means k=5	0.944	0.678	0.912	0.555	0.678	0.690	0.751	0.758	0.556	0.714
k-means k=6	0.935	0.603	0.901	0.417	0.603	0.570	0.690	0.676	0.425	0.648
k-means k=8	0.947	0.505	0.948	0.352	0.505	0.513	0.632	0.602	0.379	0.633
k-means k=10	0.948	0.467	0.946	0.282	0.467	0.435	0.603	0.533	0.308	0.594
agglo-ward k=5	0.938	0.797	0.921	0.627	0.781	0.746	0.833	0.788	0.624	0.719
agglo-average k=5	0.951	0.951	0.964	0.943	0.951	0.953	0.953	0.915	0.923	0.836
agglo-single k=5	0.587	0.997	0.401	0.998	0.585	0.572	0.532	0.574	0.006	0.012
GMM k=5	0.943	0.690	0.921	0.555	0.676	0.692	0.754	0.762	0.560	0.720
truth + 10% noise	0.906	0.906	0.928	0.826	0.906	0.874	0.910	0.833	0.798	0.686
truth, split largest	1.000	0.708	1.000	0.574	0.708	0.729	0.806	0.829	0.618	0.854
truth, merge 2 smallest	0.958	1.000	0.986	1.000	0.958	0.993	0.966	0.974	0.988	0.969
single cluster	0.583	1.000	0.399	1.000	0.583	0.571	0.532	0.571	0.000	0.000
all singletons	1.000	0.004	1.000	0.000	0.004	0.000	0.008	0.008	0.000	0.286

6. Answer: how metrics come to conflicting conclusions

1. **They measure different error types.** Purity, pair precision and BCubed precision penalise only merging, and their mirror images penalise only fragmentation. Any one-sided score can be maximised by a degenerate clustering (all singletons or a single cluster), so it orders solutions by their number of clusters as much as by their quality.
2. **They count different units.** Point-based metrics (purity, accuracy, BCubed) weight a class linearly by its size. Pair-based metrics (pair P/R/F, ARI) weight it quadratically. Errors inside the majority class therefore dominate pair metrics, and errors in small classes almost vanish from them.
3. **They match clusters to classes differently.** Purity lets several clusters claim the same class, the class F-measure lets several classes claim the same cluster, and max-matching accuracy allows only a one-to-one mapping. A fragmented solution is free under purity but pays in full under accuracy. For merges the outcome depends on the class sizes involved. Merging two equal classes is penalised less by the class F-measure (0.833) than by accuracy (0.750), because one cluster can be the best match for both classes. Merging the majority with the smallest class reverses this (class F 0.932, accuracy 0.950): the small class loses almost all of its precision, while accuracy still credits every majority point.
4. **Some correct for chance and some do not.** With a 95% majority class, putting everything in one cluster scores accuracy 0.950 and pair F1 0.949, while ARI = 0.000. The near-perfect "split majority in 2" solution scores accuracy 0.525 and pair F1 0.667, but ARI 0.162. Uncorrected metrics inherit the majority-class baseline, so they rank the useless answer higher once imbalance passes a metric-specific threshold. That threshold is $p \geq 0.70$ for accuracy and $p \geq 0.75$ for pair F1 (see Experiment 3). The same comparison on the moderately imbalanced blobs (58% majority) does not flip: the single cluster's accuracy of 0.583 is still below 0.708.
5. **They are biased by the number of clusters.** Purity tends to rise with k (on the imbalanced data it peaks at k = 13). NMI is lenient towards over-splitting and harsh on under-splitting. It ranks k-means k = 2 10 of 11, where accuracy ranks it 4, and it ranks k = 10 9, where accuracy ranks it
6. Accuracy punishes any surplus cluster in full.
7. **Imbalance amplifies all of the above.** Among the algorithmic solutions, the mean Kendall tau between the six two-sided metrics falls from 0.95 on balanced data to 0.83 on imbalanced data. A typical pair of these metrics orders 3% of solution pairs oppositely on balanced data and 9% on imbalanced data, and the average rank spread of a solution grows from 0.5 to 1.8 places.

Practical consequence: a single number cannot certify a clustering as better. Report a complementary pair that exposes both error types, such as pair or BCubed precision and recall, or purity and inverse purity. Add a chance-corrected score (ARI) and show the contingency matrix itself, which is the only view that records exactly which classes were split or merged.

References

- Amigó, E., Gonzalo, J., Artiles, J. & Verdejo, F. (2009). A comparison of extrinsic clustering evaluation metrics based on formal constraints. *Information Retrieval*, 12(4), 461–486.
- Bagga, A. & Baldwin, B. (1998). Entity-based cross-document coreferencing using the vector space model. *Proc. COLING-ACL*, 79–85.
- Hubert, L. & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2, 193–218.
- Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2, 83–97.
- Larsen, B. & Aone, C. (1999). Fast and effective text mining using linear-time document clustering. *Proc. ACM SIGKDD*, 16–22.

- Manning, C. D., Raghavan, P. & Schütze, H. (2008). *Introduction to Information Retrieval*, ch. 16. Cambridge University Press.
- Strehl, A. & Ghosh, J. (2002). Cluster ensembles — a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, 3, 583–617.
- Vinh, N. X., Epps, J. & Bailey, J. (2010). Information theoretic measures for clusterings comparison. *Journal of Machine Learning Research*, 11, 2837–2854.

Appendix A. Reproducing the results

All numbers, tables and figures in this document are produced by the code in Appendix B. From the project root:

```
pip install -r requirements.txt
python -m unittest discover -s tests -t .
python experiments/run_all.py
python experiments/build_pdf.py
```

The unit tests check every metric against hand-computed textbook values (Manning, Raghavan & Schütze, Fig. 16.4) and against scikit-learn's ARI, NMI, Fowlkes–Mallows and Rand index. Output of the test run used for this document:

```
test_ari (tests.test_metrics.TestAgainstSklearn.test_ari) ... ok
test_nmi (tests.test_metrics.TestAgainstSklearn.test_nmi) ... ok
test_pair_precision_recall_via_fowlkes_mallows
(tests.test_metrics.TestAgainstSklearn.test_pair_precision_recall_via_fowlkes_mallows) ... ok
test_rand_index (tests.test_metrics.TestAgainstSklearn.test_rand_index) ... ok
test_arbitrary_label_values (tests.test_metrics.TestContingency.test_arbitrary_label_values) ... ok
test_matrix_matches_sklearn (tests.test_metrics.TestContingency.test_matrix_matches_sklearn) ... ok
test_accuracy_matches_permutation (tests.test_metrics.TestProperties.test_accuracy_matches_permutation) ... ok
test_accuracy_penalises_extra_clusters
(tests.test_metrics.TestProperties.test_accuracy_penalises_extra_clusters) ... ok
test_class_f_bounds (tests.test_metrics.TestProperties.test_class_f_bounds) ... ok
test_degenerate_solutions (tests.test_metrics.TestProperties.test_degenerate_solutions) ... ok
test_fragmentation_keeps_purity_and_precision_perfect
(tests.test_metrics.TestProperties.test_fragmentation_keeps_purity_and_precision_perfect) ... ok
test_merging_keeps_inverse_purity_and_recall_perfect
(tests.test_metrics.TestProperties.test_merging_keeps_inverse_purity_and_recall_perfect) ... ok
test_perfect_labelling (tests.test_metrics.TestProperties.test_perfect_labelling) ... ok
test_perturbations_use_original_class_ids
(tests.test_metrics.TestProperties.test_perturbations_use_original_class_ids) ... ok
test_pair_counts (tests.test_metrics.TestTextbookValues.test_pair_counts) ... ok
test_pair_prf (tests.test_metrics.TestTextbookValues.test_pair_prf) ... ok
test_purity (tests.test_metrics.TestTextbookValues.test_purity) ... ok

----------------------------------------------------------------------
Ran 17 tests in 0.061s

OK

Status: OK
```

Appendix B. Source code

clustermetrics/__init__.py

```python
from .contingency import (
    METRIC_LABELS,
    Contingency,
    adjusted_rand_index,
    bcubed,
    class_f_measure,
    contingency_matrix,
    evaluate,
    inverse_purity,
    max_matching_accuracy,
    normalized_mutual_info,
    pair_counts,
    pair_precision_recall_f,
    purity,
)

__all__ = [
    "METRIC_LABELS",
    "Contingency",
    "adjusted_rand_index",
    "bcubed",
    "class_f_measure",
    "contingency_matrix",
    "evaluate",
    "inverse_purity",
    "max_matching_accuracy",
    "normalized_mutual_info",
    "pair_counts",
    "pair_precision_recall_f",
    "purity",
]
```

clustermetrics/contingency.py

```python
"""Contingency-matrix based external clustering evaluation.

Every metric here is computed from the contingency matrix ``M`` where
``M[i, j]`` is the number of points of true class ``i`` assigned to predicted
cluster ``j``. Row sums ``a_i`` are class sizes, column sums ``b_j`` are
cluster sizes and ``N`` is the total number of points.
"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from scipy.optimize import linear_sum_assignment


@dataclass(frozen=True)
class Contingency:
    matrix: np.ndarray
    classes: np.ndarray
    clusters: np.ndarray

    @property
    def n(self) -> int:
        return int(self.matrix.sum())

    @property
    def class_sizes(self) -> np.ndarray:
        return self.matrix.sum(axis=1)

    @property
    def cluster_sizes(self) -> np.ndarray:
        return self.matrix.sum(axis=0)

    def to_frame(self):
        import pandas as pd
```

```
        return pd.DataFrame(
            self.matrix,
            index=[f"class {c}" for c in self.classes],
            columns=[f"cluster {k}" for k in self.clusters],
        )


def contingency_matrix(y_true, y_pred) -> Contingency:
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    if y_true.shape != y_pred.shape or y_true.ndim != 1:
        raise ValueError("y_true and y_pred must be 1-D arrays of equal length")
    if y_true.size == 0:
        raise ValueError("cannot evaluate an empty labelling")

    classes, class_idx = np.unique(y_true, return_inverse=True)
    clusters, cluster_idx = np.unique(y_pred, return_inverse=True)
    matrix = np.zeros((classes.size, clusters.size), dtype=np.int64)
    np.add.at(matrix, (class_idx, cluster_idx), 1)
    return Contingency(matrix, classes, clusters)


def _as_contingency(y_true_or_c, y_pred=None) -> Contingency:
    if isinstance(y_true_or_c, Contingency):
        return y_true_or_c
    return contingency_matrix(y_true_or_c, y_pred)


def _comb2(x) -> np.ndarray | float:
    x = np.asarray(x, dtype=np.float64)
    return x * (x - 1.0) / 2.0


def _safe_div(num: float, den: float, empty: float) -> float:
    return float(num / den) if den > 0 else empty


def _f(p: float, r: float) -> float:
    return 2.0 * p * r / (p + r) if (p + r) > 0 else 0.0


# --- Set-matching metrics ----------------------------------------------------


def purity(y_true, y_pred=None) -> float:
    """Fraction of points belonging to the majority class of their cluster."""
    c = _as_contingency(y_true, y_pred)
    return float(c.matrix.max(axis=0).sum() / c.n)


def inverse_purity(y_true, y_pred=None) -> float:
    """Fraction of points belonging to the dominant cluster of their class."""
    c = _as_contingency(y_true, y_pred)
    return float(c.matrix.max(axis=1).sum() / c.n)


def max_matching_accuracy(y_true, y_pred=None) -> tuple[float, dict]:
    """Accuracy under the best one-to-one class/cluster mapping (Hungarian).

    When the number of clusters differs from the number of classes, surplus
    clusters (or classes) stay unmatched and all their points count as errors.
    Returns the accuracy and the ``{cluster: class}`` mapping.
    """
    c = _as_contingency(y_true, y_pred)
    rows, cols = linear_sum_assignment(c.matrix, maximize=True)
    mapping = {c.clusters[j].item(): c.classes[i].item() for i, j in zip(rows, cols)}
    return float(c.matrix[rows, cols].sum() / c.n), mapping


def class_f_measure(y_true, y_pred=None) -> float:
    """Larsen & Aone (1999) F-measure.

    For each class, take the best F1 over all clusters, then average weighted
    by class size. A cluster may be the best match for several classes.
    """
    c = _as_contingency(y_true, y_pred)
    m = c.matrix.astype(np.float64)
```

```python
    a = c.class_sizes[:, None].astype(np.float64)
    b = c.cluster_sizes[None, :].astype(np.float64)
    precision = m / b
    recall = m / a
    with np.errstate(invalid="ignore", divide="ignore"):
        f = np.where(m > 0, 2 * precision * recall / (precision + recall), 0.0)
    return float((c.class_sizes * f.max(axis=1)).sum() / c.n)


# --- Pair-counting metrics ---------------------------------------------------


def pair_counts(y_true, y_pred=None) -> dict[str, float]:
    """Classify all N(N-1)/2 point pairs.

    TP: same class & same cluster; FP: different class, same cluster;
    FN: same class, different cluster; TN: different class & different cluster.
    """
    c = _as_contingency(y_true, y_pred)
    same_both = float(_comb2(c.matrix).sum())
    same_cluster = float(_comb2(c.cluster_sizes).sum())
    same_class = float(_comb2(c.class_sizes).sum())
    total = float(_comb2(c.n))
    tp = same_both
    fp = same_cluster - tp
    fn = same_class - tp
    tn = total - tp - fp - fn
    return {"tp": tp, "fp": fp, "fn": fn, "tn": tn}


def pair_precision_recall_f(y_true, y_pred=None) -> tuple[float, float, float]:
    """Pairwise precision, recall and F1.

    Precision: of pairs put together, the fraction that share a class.
    Recall: of pairs sharing a class, the fraction put together.
    0/0 precision (all singletons) is defined as 1 since no wrong pair is
    asserted; 0/0 recall (all classes singletons) is defined likewise.
    """
    pc = pair_counts(y_true, y_pred)
    p = _safe_div(pc["tp"], pc["tp"] + pc["fp"], 1.0)
    r = _safe_div(pc["tp"], pc["tp"] + pc["fn"], 1.0)
    return p, r, _f(p, r)


def bcubed(y_true, y_pred=None) -> tuple[float, float, float]:
    """BCubed precision, recall and F1 (Bagga & Baldwin 1998; Amigo et al. 2009)."""
    c = _as_contingency(y_true, y_pred)
    m = c.matrix.astype(np.float64)
    sq = m**2
    p = float((sq / c.cluster_sizes[None, :]).sum() / c.n)
    r = float((sq / c.class_sizes[:, None]).sum() / c.n)
    return p, r, _f(p, r)


def adjusted_rand_index(y_true, y_pred=None) -> float:
    c = _as_contingency(y_true, y_pred)
    index = float(_comb2(c.matrix).sum())
    sum_a = float(_comb2(c.class_sizes).sum())
    sum_b = float(_comb2(c.cluster_sizes).sum())
    expected = sum_a * sum_b / float(_comb2(c.n)) if c.n > 1 else 0.0
    max_index = 0.5 * (sum_a + sum_b)
    if np.isclose(max_index, expected):
        return 1.0
    return (index - expected) / (max_index - expected)


# --- Information-theoretic ---------------------------------------------------


def _entropy(counts: np.ndarray) -> float:
    p = counts[counts > 0] / counts.sum()
    return float(-(p * np.log(p)).sum())


def normalized_mutual_info(y_true, y_pred=None) -> float:
    """NMI with arithmetic-mean normalisation (sklearn's default)."""
    c = _as_contingency(y_true, y_pred)
    if c.classes.size == c.clusters.size == 1:
```

```
        return 1.0
    m = c.matrix.astype(np.float64)
    n = c.n
    nz = m > 0
    outer = np.outer(c.class_sizes, c.cluster_sizes).astype(np.float64)
    mi = float((m[nz] / n * np.log(m[nz] * n / outer[nz])).sum())
    h_true = _entropy(c.class_sizes.astype(np.float64))
    h_pred = _entropy(c.cluster_sizes.astype(np.float64))
    denom = 0.5 * (h_true + h_pred)
    if denom <= 0:
        return 1.0
    return max(mi, 0.0) / denom


# --- Bundle ------------------------------------------------------------------

METRIC_LABELS = {
    "purity": "Purity",
    "inverse_purity": "Inverse purity",
    "accuracy": "Max-matching accuracy",
    "pair_precision": "Pair precision",
    "pair_recall": "Pair recall",
    "pair_f1": "Pair F1",
    "class_f": "Class F-measure",
    "bcubed_precision": "BCubed precision",
    "bcubed_recall": "BCubed recall",
    "bcubed_f1": "BCubed F1",
    "ari": "ARI",
    "nmi": "NMI",
}


def evaluate(y_true, y_pred=None) -> dict[str, float]:
    """All metrics for one labelling; higher is better for every entry."""
    c = _as_contingency(y_true, y_pred)
    pp, pr, pf = pair_precision_recall_f(c)
    bp, br, bf = bcubed(c)
    return {
        "purity": purity(c),
        "inverse_purity": inverse_purity(c),
        "accuracy": max_matching_accuracy(c)[0],
        "pair_precision": pp,
        "pair_recall": pr,
        "pair_f1": pf,
        "class_f": class_f_measure(c),
        "bcubed_precision": bp,
        "bcubed_recall": br,
        "bcubed_f1": bf,
        "ari": adjusted_rand_index(c),
        "nmi": normalized_mutual_info(c),
        "n_clusters": int(c.clusters.size),
    }
```

clustermetrics/perturb.py

```
"""Controlled corruptions of a ground-truth labelling.

Each function returns a new predicted labelling so that the exact kind and
amount of error is known, which lets us attribute metric behaviour to a
specific failure mode. Class ids passed in refer to the original values in
``y_true``; outputs are relabelled to ``0..k-1``.
"""

from __future__ import annotations

import numpy as np


def _relabel(y) -> np.ndarray:
    return np.unique(np.asarray(y), return_inverse=True)[1]


def _encode(y_true):
    classes, codes = np.unique(np.asarray(y_true), return_inverse=True)

    def code_of(ids) -> np.ndarray:
```

```
        ids = np.atleast_1d(np.asarray(ids))
        pos = np.searchsorted(classes, ids)
        if np.any(pos >= classes.size) or np.any(classes[np.minimum(pos, classes.size - 1)] != ids):
            raise ValueError(f"unknown class id(s) {ids.tolist()}; classes are {classes.tolist()}")
        return pos

    return codes.astype(np.int64), code_of


def fragment(y_true, classes, pieces: int, rng=None) -> np.ndarray:
    """Split every class in ``classes`` into ``pieces`` near-equal clusters."""
    rng = np.random.default_rng(rng)
    y, code_of = _encode(y_true)
    pred = y.copy()
    next_label = int(y.max()) + 1
    for cls in code_of(classes):
        idx = np.flatnonzero(y == cls)
        if pieces > idx.size:
            raise ValueError(f"cannot split a class of {idx.size} points into {pieces} pieces")
        rng.shuffle(idx)
        for part in np.array_split(idx, pieces)[1:]:
            pred[part] = next_label
            next_label += 1
    return _relabel(pred)


def fragment_all(y_true, pieces: int, rng=None) -> np.ndarray:
    return fragment(y_true, np.unique(y_true), pieces, rng)


def merge(y_true, groups) -> np.ndarray:
    """Merge classes; ``groups`` is an iterable of class-id collections."""
    y, code_of = _encode(y_true)
    pred = y.copy()
    for group in groups:
        codes = code_of(list(group))
        pred[np.isin(y, codes)] = codes[0]
    return _relabel(pred)


def label_noise(y_true, rate: float, rng=None) -> np.ndarray:
    """Independently, with probability ``rate``, move each point to a random other label."""
    rng = np.random.default_rng(rng)
    y = _relabel(y_true)
    k = int(y.max()) + 1
    pred = y.copy()
    flip = rng.random(y.size) < rate
    shift = rng.integers(1, k, size=flip.sum()) if k > 1 else 0
    pred[flip] = (y[flip] + shift) % max(k, 1)
    return pred


def single_cluster(y_true) -> np.ndarray:
    return np.zeros(len(y_true), dtype=np.int64)


def singletons(y_true) -> np.ndarray:
    return np.arange(len(y_true), dtype=np.int64)
```

clustermetrics/ranking.py

```
"""Tools for comparing how different metrics order the same solutions."""

from __future__ import annotations

import itertools

import numpy as np
import pandas as pd
from scipy.stats import kendalltau


def rank_solutions(scores: pd.DataFrame) -> pd.DataFrame:
    """Rank solutions (rows) per metric (columns); 1 = best, ties share min rank."""
    return scores.rank(axis=0, ascending=False, method="min").astype(int)

```

```python

def kendall_matrix(scores: pd.DataFrame) -> pd.DataFrame:
    cols = scores.columns
    out = pd.DataFrame(np.eye(len(cols)), index=cols, columns=cols)
    for a, b in itertools.combinations(cols, 2):
        tau = kendalltau(scores[a], scores[b]).statistic
        out.loc[a, b] = out.loc[b, a] = tau
    return out


def disagreement_rate(scores: pd.DataFrame, tol: float = 1e-12) -> pd.DataFrame:
    """Fraction of solution pairs that two metrics order in opposite directions.

    Pairs tied under either metric are excluded, so 0 means the metrics never
    contradict each other and 1 means they always do.
    """
    cols = scores.columns
    values = scores.to_numpy()
    i, j = np.triu_indices(len(scores), k=1)
    diffs = values[i] - values[j]
    signs = np.where(np.abs(diffs) <= tol, 0, np.sign(diffs))
    out = pd.DataFrame(np.zeros((len(cols), len(cols))), index=cols, columns=cols)
    for a, b in itertools.combinations(range(len(cols)), 2):
        both = (signs[:, a] != 0) & (signs[:, b] != 0)
        rate = float((signs[both, a] != signs[both, b]).mean()) if both.any() else 0.0
        out.iloc[a, b] = out.iloc[b, a] = rate
    return out


def pairwise_conflicts(scores: pd.DataFrame, metric_a: str, metric_b: str, tol: float = 1e-12):
    """List solution pairs (x, y) where metric_a prefers x but metric_b prefers y."""
    rows = []
    for x, y in itertools.permutations(scores.index, 2):
        da = scores.at[x, metric_a] - scores.at[y, metric_a]
        db = scores.at[x, metric_b] - scores.at[y, metric_b]
        if da > tol and db < -tol:
            rows.append((x, y, da, db))
    return pd.DataFrame(rows, columns=["preferred_by_a", "preferred_by_b", "delta_a", "delta_b"])
```

experiments/run_all.py

```python
"""Run every experiment and write figures, CSVs and results/report.md.

Usage:  python experiments/run_all.py
"""

from __future__ import annotations

import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.cluster import AgglomerativeClustering, KMeans
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture

from clustermetrics import METRIC_LABELS, contingency_matrix, evaluate, max_matching_accuracy
from clustermetrics import perturb
from clustermetrics.ranking import (
    disagreement_rate,
    kendall_matrix,
    pairwise_conflicts,
    rank_solutions,
)

OUT = ROOT / "results"
FIG = OUT / "figures"
SEED = 0
```

```

CENTERS = np.array([[0, 0], [6, 0], [0, 6], [6, 6], [3, 3]], dtype=float)
CLUSTER_STD = 1.3
DATASETS = {
    "balanced": [240, 240, 240, 240, 240],
    "imbalanced": [700, 250, 120, 80, 50],
}
TRUE_K = len(CENTERS)

ONE_SIDED = ["purity", "inverse_purity", "pair_precision", "pair_recall"]
TWO_SIDED = ["accuracy", "pair_f1", "class_f", "bcubed_f1", "ari", "nmi"]
RANKED = ONE_SIDED + TWO_SIDED
SWEEP_METRICS = ["purity", "inverse_purity", "accuracy", "pair_f1", "class_f", "bcubed_f1", "ari", "nmi"]


# --- helpers ---------------------------------------------------------------


def label(m: str) -> str:
    return METRIC_LABELS.get(m, m)


def md_table(df: pd.DataFrame, fmt: str = "{:.3f}", index: bool = True) -> str:
    header = ([str(df.index.name or "")] if index else []) + [str(c) for c in df.columns]
    lines = ["| " + " | ".join(header) + " |", "|" + "|".join(["---"] * len(header)) + "|"]
    for idx, row in df.iterrows():
        cells = [str(idx)] if index else []
        for v in row:
            if isinstance(v, (float, np.floating)):
                cells.append("–" if np.isnan(v) else fmt.format(v))
            else:
                cells.append(str(v))
        lines.append("| " + " | ".join(cells) + " |")
    return "\n".join(lines)


def make_dataset(sizes):
    return make_blobs(
        n_samples=sizes, centers=CENTERS, cluster_std=CLUSTER_STD, random_state=SEED
    )


def argmax_all(values: pd.Series, tol: float = 1e-9) -> list:
    return list(values.index[values >= values.max() - tol])


def heatmap(ax, data: pd.DataFrame, fmt: str, cmap: str, vmin=None, vmax=None):
    im = ax.imshow(data.to_numpy(dtype=float), cmap=cmap, vmin=vmin, vmax=vmax, aspect="auto")
    ax.set_xticks(range(data.shape[1]), [label(c) for c in data.columns], rotation=45, ha="right")
    ax.set_yticks(range(data.shape[0]), [label(r) for r in data.index])
    for i in range(data.shape[0]):
        for j in range(data.shape[1]):
            v = data.iat[i, j]
            if pd.isna(v):
                ax.text(j, i, "–", ha="center", va="center", fontsize=8)
                continue
            r, g, b, _ = im.cmap(im.norm(v))
            color = "white" if 0.299 * r + 0.587 * g + 0.114 * b < 0.5 else "black"
            ax.text(j, i, fmt.format(v), ha="center", va="center", fontsize=8, color=color)
    return im


# --- Experiment 0: data and a worked contingency matrix ------------------------


def exp_datasets(data):
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.6))
    for ax, name in zip(axes[:2], data):
        X, y = data[name]
        ax.scatter(X[:, 0], X[:, 1], c=y, s=5, cmap="tab10")
        ax.set_title(f"{name} ground truth, sizes {DATASETS[name]}")
    X, y = data["imbalanced"]
    pred = KMeans(TRUE_K, n_init=10, random_state=SEED).fit_predict(X)
    axes[2].scatter(X[:, 0], X[:, 1], c=pred, s=5, cmap="tab10")
    axes[2].set_title(f"imbalanced, k-means k={TRUE_K}")
    for ax in axes:
        ax.set_aspect("equal")
    fig.tight_layout()
```

```
    fig.savefig(FIG / "datasets.png", dpi=130)
    plt.close(fig)

    pred8 = KMeans(8, n_init=10, random_state=SEED).fit_predict(X)
    c = contingency_matrix(y, pred8)
    frame = c.to_frame()
    frame["class size"] = c.class_sizes
    frame.loc["cluster size"] = list(c.cluster_sizes) + [c.n]
    frame.loc["majority class"] = list(c.classes[c.matrix.argmax(axis=0)]) + [""]
    acc, mapping = max_matching_accuracy(c)
    scores = evaluate(c)
    return {
        "frame": frame,
        "mapping": mapping,
        "acc": acc,
        "scores": scores,
        "frame_k5": contingency_matrix(y, pred).to_frame(),
        "frame_k4": contingency_matrix(
            y, KMeans(TRUE_K - 1, n_init=10, random_state=SEED).fit_predict(X)
        ).to_frame(),
    }


# --- Experiment 1: number of clusters ---------------------------------------------


def exp_vary_k(data, ks=range(1, 16)):
    rows = []
    for name, (X, y) in data.items():
        for k in ks:
            pred = KMeans(k, n_init=10, random_state=SEED).fit_predict(X)
            rows.append({"dataset": name, "k": k, **evaluate(y, pred)})
    df = pd.DataFrame(rows)
    df.to_csv(OUT / "exp1_vary_k.csv", index=False)

    best = pd.DataFrame(index=pd.Index([label(m) for m in RANKED], name="Metric"))
    for name in data:
        sub = df[df.dataset == name].set_index("k")
        best[f"best k ({name})"] = [", ".join(map(str, argmax_all(sub[m]))) for m in RANKED]

    fig, axes = plt.subplots(2, 2, figsize=(14, 9), sharex=True)
    for r, name in enumerate(data):
        sub = df[df.dataset == name]
        for c, (group, title) in enumerate([(ONE_SIDED, "one-sided"), (TWO_SIDED, "two-sided")]):
            ax = axes[r, c]
            for m in group:
                ax.plot(sub.k, sub[m], marker="o", ms=3, label=label(m))
            ax.axvline(TRUE_K, color="grey", ls="--", lw=1)
            ax.set_title(f"{name} data: {title} metrics")
            ax.set_ylim(-0.02, 1.02)
            ax.grid(alpha=0.3)
            ax.legend(fontsize=8, loc="lower right" if c == 0 else "upper right")
    for ax in axes[1]:
        ax.set_xlabel("number of k-means clusters (dashed = true number of classes)")
    fig.tight_layout()
    fig.savefig(FIG / "exp1_vary_k.png", dpi=130)
    plt.close(fig)
    return df, best


# --- Experiment 2: fragmentation and merging ---------------------------------------


def exp_frag_merge():
    y = np.repeat(np.arange(8), 100)
    frag = pd.DataFrame(
        [{"pieces": m, **evaluate(y, perturb.fragment_all(y, m, rng=SEED))} for m in range(1, 9)]
    )
    merge = pd.DataFrame(
        [{"merged": g, **evaluate(y, perturb.merge(y, [range(g)]))} for g in range(1, 9)]
    )
    frag.to_csv(OUT / "exp2_fragmentation.csv", index=False)
    merge.to_csv(OUT / "exp2_merging.csv", index=False)

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    for ax, df, x, title in [
        (axes[0], frag, "pieces", "Fragmentation: every class split into m clusters"),
        (axes[1], merge, "merged", "Merging: g of the 8 classes merged into one cluster"),
```

```
    ]:
        for m in RANKED:
            ax.plot(df[x], df[m], marker="o", ms=3, label=label(m))
        ax.set_title(title)
        ax.set_xlabel(x)
        ax.set_ylim(-0.02, 1.02)
        ax.grid(alpha=0.3)
    axes[1].legend(fontsize=8, loc="lower left")
    fig.tight_layout()
    fig.savefig(FIG / "exp2_frag_merge.png", dpi=130)
    plt.close(fig)

    # Single errors of comparable size, on balanced vs imbalanced truths.
    y_bal = np.repeat(np.arange(4), 250)
    y_imb = np.repeat(np.arange(4), [700, 150, 100, 50])
    scenarios = {
        "balanced: split one class in 2": (y_bal, perturb.fragment(y_bal, [0], 2, rng=SEED)),
        "balanced: merge two classes": (y_bal, perturb.merge(y_bal, [[0, 1]])),
        "imbalanced: split majority in 2": (y_imb, perturb.fragment(y_imb, [0], 2, rng=SEED)),
        "imbalanced: merge two smallest": (y_imb, perturb.merge(y_imb, [[2, 3]])),
        "imbalanced: merge majority + smallest": (y_imb, perturb.merge(y_imb, [[0, 3]])),
    }
    single = pd.DataFrame({k: evaluate(*v) for k, v in scenarios.items()}).T[RANKED]
    single.index.name = "Scenario"
    single.to_csv(OUT / "exp2_single_errors.csv")
    return frag, merge, single


# --- Experiment 3: class imbalance sweep ------------------------------------------------


def imbalanced_truth(p: float, n: int = 1000, n_minor: int = 3) -> np.ndarray:
    major = int(round(p * n))
    minors = [len(a) for a in np.array_split(np.arange(n - major), n_minor)]
    return np.repeat(np.arange(n_minor + 1), [major] + minors)


IMB_SOLUTIONS = {
    "single cluster": lambda y: perturb.single_cluster(y),
    "split majority in 2": lambda y: perturb.fragment(y, [0], 2, rng=SEED),
    "merge all minorities": lambda y: perturb.merge(y, [[1, 2, 3]]),
    "20% random noise": lambda y: perturb.label_noise(y, 0.2, rng=SEED),
}


def exp_imbalance(fractions=np.round(np.arange(0.25, 0.951, 0.05), 2)):
    rows = []
    for p in fractions:
        y = imbalanced_truth(p)
        for name, make in IMB_SOLUTIONS.items():
            rows.append({"majority_fraction": p, "solution": name, **evaluate(y, make(y))})
    df = pd.DataFrame(rows)
    df.to_csv(OUT / "exp3_imbalance.csv", index=False)

    fig, axes = plt.subplots(2, 4, figsize=(18, 8), sharex=True)
    for ax, m in zip(axes.flat, SWEEP_METRICS):
        for name in IMB_SOLUTIONS:
            sub = df[df.solution == name]
            ax.plot(sub.majority_fraction, sub[m], marker="o", ms=3, label=name)
        ax.set_title(label(m))
        ax.set_ylim(-0.05, 1.02)
        ax.grid(alpha=0.3)
    for ax in axes[1]:
        ax.set_xlabel("majority class fraction")
    axes[0, 0].legend(fontsize=8)
    fig.tight_layout()
    fig.savefig(FIG / "exp3_imbalance.png", dpi=130)
    plt.close(fig)

    # Where does the degenerate single cluster start beating a near-perfect split?
    cross = {}
    for m in SWEEP_METRICS:
        a = df[df.solution == "single cluster"].set_index("majority_fraction")[m]
        b = df[df.solution == "split majority in 2"].set_index("majority_fraction")[m]
        wins = (a - b) >= -1e-12
        if wins.all():
            cross[label(m)] = "always"
        elif not wins.any():
```

```
            cross[label(m)] = "never"
        else:
            cross[label(m)] = f"p ≥ {wins[wins].index.min():.2f}"
    cross = pd.DataFrame({"single cluster ≥ split-majority when": cross})
    cross.index.name = "Metric"

    winners = {}
    for p in [0.25, 0.5, 0.75, 0.95]:
        sub = df[np.isclose(df.majority_fraction, p)].set_index("solution")
        winners[f"p = {p:.2f}"] = [" / ".join(argmax_all(sub[m])) for m in SWEEP_METRICS]
    winners = pd.DataFrame(winners, index=pd.Index([label(m) for m in SWEEP_METRICS], name="Metric"))
    return df, cross, winners


# --- Experiment 4: ranking comparison ----------------------------------------------------


def candidate_solutions(X, y) -> dict[str, np.ndarray]:
    sizes = np.bincount(y)
    largest = int(sizes.argmax())
    smallest_two = list(np.argsort(sizes)[:2])
    sols = {}
    for k in [2, 3, 4, 5, 6, 8, 10]:
        sols[f"k-means k={k}"] = KMeans(k, n_init=10, random_state=SEED).fit_predict(X)
    for linkage in ["ward", "average", "single"]:
        sols[f"agglo-{linkage} k={TRUE_K}"] = AgglomerativeClustering(
            TRUE_K, linkage=linkage
        ).fit_predict(X)
    sols[f"GMM k={TRUE_K}"] = GaussianMixture(TRUE_K, random_state=SEED).fit(X).predict(X)
    sols["truth + 10% noise"] = perturb.label_noise(y, 0.1, rng=SEED)
    sols["truth, split largest"] = perturb.fragment(y, [largest], 2, rng=SEED)
    sols["truth, merge 2 smallest"] = perturb.merge(y, [smallest_two])
    sols["single cluster"] = perturb.single_cluster(y)
    sols["all singletons"] = perturb.singletons(y)
    return sols


def is_algorithmic(solution: str) -> bool:
    return not (solution.startswith("truth") or solution in ("single cluster", "all singletons"))


def analyse_pool(metric_scores: pd.DataFrame) -> dict:
    return {
        "metric_scores": metric_scores,
        "ranks": rank_solutions(metric_scores),
        "tau": kendall_matrix(metric_scores),
        "dis": disagreement_rate(metric_scores),
        "winners": pd.DataFrame(
            {"Top-ranked solution(s)": [", ".join(argmax_all(metric_scores[m])) for m in RANKED]},
            index=pd.Index([label(m) for m in RANKED], name="Metric"),
        ),
    }


def pool_summary(pool: dict) -> dict:
    out = {}
    for group_name, group in [("all 10 metrics", RANKED), ("6 two-sided metrics", TWO_SIDED)]:
        t = pool["tau"].loc[group, group].to_numpy()
        d = pool["dis"].loc[group, group].to_numpy()
        iu = np.triu_indices(len(group), 1)
        out[f"mean Kendall tau ({group_name})"] = np.nanmean(t[iu])
        out[f"min Kendall tau ({group_name})"] = np.nanmin(t[iu])
        out[f"mean disagreement rate ({group_name})"] = np.mean(d[iu])
    ranks = pool["ranks"][TWO_SIDED]
    out["mean rank spread across two-sided metrics"] = float((ranks.max(axis=1) - ranks.min(axis=1)).mean())
    return out


def exp_rankings(data):
    results = {}
    for name, (X, y) in data.items():
        sols = candidate_solutions(X, y)
        scores = pd.DataFrame({s: evaluate(y, p) for s, p in sols.items()}).T
        scores.index.name = "Solution"
        scores.to_csv(OUT / f"exp4_scores_{name}.csv")
        metric_scores = scores[RANKED].astype(float)
        full = analyse_pool(metric_scores)
        algo = analyse_pool(metric_scores.loc[[s for s in metric_scores.index if is_algorithmic(s)]])
```

```
        for pool_name, pool in [("all", full), ("algorithms", algo)]:
            pool["ranks"].to_csv(OUT / f"exp4_ranks_{name}_{pool_name}.csv")
            pool["tau"].to_csv(OUT / f"exp4_kendall_{name}_{pool_name}.csv")
            pool["dis"].to_csv(OUT / f"exp4_disagreement_{name}_{pool_name}.csv")
        results[name] = {"scores": scores, "all": full, "algorithms": algo}

    for pool_name in ["all", "algorithms"]:
        fig, axes = plt.subplots(1, 2, figsize=(17, 7.5 if pool_name == "all" else 6.5))
        for ax, name in zip(axes, data):
            heatmap(ax, results[name][pool_name]["ranks"], "{:d}", "viridis_r")
            ax.set_title(f"{name} data: rank per metric (1 = best)")
        fig.tight_layout()
        fig.savefig(FIG / f"exp4_ranks_{pool_name}.png", dpi=130)
        plt.close(fig)

        fig, axes = plt.subplots(1, 2, figsize=(19, 8), layout="constrained")
        for ax, name in zip(axes, data):
            im = heatmap(ax, results[name][pool_name]["tau"], "{:.2f}", "RdBu", -1, 1)
            ax.set_title(f"{name} data: Kendall tau between metric rankings")
        fig.colorbar(im, ax=axes, shrink=0.8)
        fig.savefig(FIG / f"exp4_kendall_{pool_name}.png", dpi=130, bbox_inches="tight")
        plt.close(fig)

    summary = pd.DataFrame(
        {
            f"{name} / {pool_name}": pool_summary(results[name][pool_name])
            for pool_name in ["all", "algorithms"]
            for name in data
        }
    )
    summary.index.name = "Statistic"
    summary.to_csv(OUT / "exp4_summary.csv")
    return results, summary


def top_conflicts(metric_scores, a, b, n=3):
    c = pairwise_conflicts(metric_scores, a, b)
    if c.empty:
        return c
    c["strength"] = np.minimum(c.delta_a.abs(), c.delta_b.abs())
    return c.sort_values("strength", ascending=False).head(n)


# --- Report ------------------------------------------------------------------


def fmt_scores(df, cols=RANKED):
    out = df[cols].astype(float).copy()
    out.columns = [label(c) for c in cols]
    return out


def write_report(ex0, e1, e2, e3, e4):
    _, best_k = e1
    frag, merge, single = e2
    imb, cross, imb_winners = e3
    res, summary = e4
    imb_res = res["imbalanced"]
    ms = imb_res["algorithms"]["metric_scores"]
    algo_ranks = imb_res["algorithms"]["ranks"]

    conflict_blocks = []
    for a, b in [("accuracy", "nmi"), ("pair_f1", "ari"), ("bcubed_f1", "accuracy"), ("class_f", "ari")]:
        c = top_conflicts(ms, a, b)
        if c.empty:
            conflict_blocks.append(f"- **{label(a)} vs {label(b)}**: no reversals on this pool.")
            continue
        lines = []
        for _, r in c.iterrows():
            x, y = r.preferred_by_a, r.preferred_by_b
            lines.append(
                f"  - {label(a)} prefers *{x}* ({ms.at[x, a]:.3f} vs {ms.at[y, a]:.3f}); "
                f"{label(b)} prefers *{y}* ({ms.at[y, b]:.3f} vs {ms.at[x, b]:.3f})"
            )
        n_all = len(pairwise_conflicts(ms, a, b))
        conflict_blocks.append(
            f"- **{label(a)} vs {label(b)}** reverse the order of {n_all} solution pairs, e.g.\n"
            + "\n".join(lines)
```

```
        )

    single_cluster = imb_res["scores"].loc["single cluster"]
    split_largest = imb_res["scores"].loc["truth, split largest"]
    s_bal, s_imb = summary["balanced / algorithms"], summary["imbalanced / algorithms"]
    tau_key, dis_key = "mean Kendall tau (6 two-sided metrics)", "mean disagreement rate (6 two-sided metrics)"
    spread_key = "mean rank spread across two-sided metrics"
    p95 = imb[np.isclose(imb.majority_fraction, 0.95)].set_index("solution")
    k1 = e1[0][e1[0].dataset == "imbalanced"].set_index("k")
    n_algo = len(algo_ranks)

    def rank_of(sol, m):
        return int(algo_ranks.at[sol, m])

    best_k_imb = {m: argmax_all(k1[m]) for m in RANKED}

    def by_solution(pick) -> str:
        groups: dict[str, list[str]] = {}
        for m in TWO_SIDED:
            groups.setdefault(pick(ms[m]), []).append(label(m))
        return "; ".join(f"*{s}* by {', '.join(ms_)}" for s, ms_ in groups.items())

    full_ms = imb_res["all"]["metric_scores"]
    full_top = {s for m in TWO_SIDED for s in argmax_all(full_ms[m])}
    if len(full_top) == 1:
        top = next(iter(full_top))
        full_pool_lead = (
            f"One controlled corruption, *{top}*, is closer to the truth than any algorithm "
            "output, and every two-sided metric ranks it first. The other corruptions do not "
            "dominate: *truth + 10% noise*, for example, loses to the best algorithms on "
            "every two-sided metric."
        )
    else:
        full_pool_lead = (
            "The two-sided metrics do not even agree on the best solution in the full pool."
        )
    biggest = algo_ranks[TWO_SIDED].max(axis=1) - algo_ranks[TWO_SIDED].min(axis=1)
    widest = biggest.idxmax()
    widest_lo = algo_ranks.loc[widest, TWO_SIDED].idxmin()
    widest_hi = algo_ranks.loc[widest, TWO_SIDED].idxmax()
    merge_bal = single.loc["balanced: merge two classes"]
    merge_mix = single.loc["imbalanced: merge majority + smallest"]
    best_algo = by_solution(lambda s: s.idxmax())
    worst_algo = by_solution(lambda s: s.idxmin())
    p95_single, p95_split = p95.loc["single cluster"], p95.loc["split majority in 2"]

    report = f"""# When clustering metrics disagree

Research question: **how can evaluation metrics produce conflicting conclusions about
clustering quality?**

## Abstract

External clustering metrics compress the contingency matrix between predicted clusters and
true classes into a single number, and each compression encodes a different notion of
error. We implement purity, inverse purity, maximum-matching accuracy, pair-counting
precision, recall and F1, the class F-measure, BCubed, ARI and NMI from one contingency
matrix. We then evaluate them under controlled fragmentation and merging, a class-imbalance
sweep, k-means with k = 1…15, and a ranking of 16 candidate solutions on balanced and
imbalanced data.

One-sided metrics are maximised by degenerate clusterings. On imbalanced data, purity
peaks at k = {", ".join(map(str, best_k_imb["purity"]))} and pair recall at
k = {", ".join(map(str, best_k_imb["pair_recall"]))}. Metrics without chance correction
prefer a single all-inclusive cluster over a near-perfect solution once the majority class
exceeds a metric-specific share of the data: {cross.iloc[:, 0][label("accuracy")]} for
accuracy and {cross.iloc[:, 0][label("bcubed_f1")]} for BCubed F1. ARI and NMI never do.
Among {n_algo} solutions produced by clustering algorithms, the mean Kendall rank
correlation between six two-sided metrics drops from {s_bal[tau_key]:.2f} on balanced data
to {s_imb[tau_key]:.2f} on imbalanced data.

Conflicts arise from four design choices hidden inside each metric: which error type it
penalises, whether it counts points or pairs, how it matches clusters to classes, and
whether it corrects for chance. Class imbalance and a wrong number of clusters amplify all
four.

## Introduction and method
```

```
Clustering is unsupervised, so when ground-truth labels exist the natural evaluation is to
compare the predicted partition with the true one. Cluster identifiers are arbitrary,
however, so every external metric first builds the **contingency matrix** (true classes ×
predicted clusters) and then summarises it. The research question is how the choice of
summary can change the verdict.

Two failure modes are central:

- **Fragmentation:** a single true class is split across several clusters.
- **Merging:** several true classes share one cluster.

**Class imbalance** matters because it changes how much each class contributes to a
point-weighted or pair-weighted score.

**Data.** The data are two-dimensional Gaussian blobs with 5 classes around fixed centres
(standard deviation {CLUSTER_STD}, so neighbouring classes overlap). The balanced version
has 5 × 240 points; the imbalanced one has class sizes {DATASETS["imbalanced"]}. The
controlled experiments use synthetic label vectors so that the type and size of every
error is known exactly.

**Clusterings.** k-means (k = 1…15), agglomerative clustering (ward, average and single
linkage), a Gaussian mixture model, and constructed corruptions of the truth (fragmenting a
class, merging classes, random label noise, one cluster, all singletons).

**Comparison of rankings.** For each metric, the solutions are ranked from best to worst.
Agreement between two metrics is measured by Kendall's tau over solution scores and by
the *disagreement rate*: the fraction of solution pairs, among those not tied under
either metric, that the two metrics order in opposite directions.

The implementation is verified by unit tests against textbook values and scikit-learn
(Appendix A). Every table and figure is regenerated by `python experiments/run_all.py`.

## 1. Metrics, all derived from the contingency matrix

Let `n_ij` be the number of points of true class *i* in predicted cluster *j*, `a_i` the
class sizes, `b_j` the cluster sizes and `N` the number of points.

| Metric | Definition | Error it ignores |
|---|---|---|
| Purity | Σ_j max_i n_ij / N | fragmentation (all singletons = 1) |
| Inverse purity | Σ_i max_j n_ij / N | merging (one cluster = 1) |
| Max-matching accuracy | best one-to-one class↔cluster assignment (Hungarian), matched points / N | none, but surplus clusters count fully as errors |
| Pair precision | pairs together in both / pairs together in prediction | fragmentation |
| Pair recall | pairs together in both / pairs together in truth | merging |
| Pair F1 | harmonic mean of pair precision and recall | weights classes by size² |
| Class F-measure | Σ_i (a_i/N) max_j F(i, j) (Larsen & Aone) | one cluster may serve several classes |
| BCubed F1 | per-point precision / recall, then F1 | — |
| ARI | Rand index corrected for chance | — |
| NMI | mutual information / mean entropy | mild bias towards more clusters |

### Worked contingency matrix (imbalanced data, k-means with k = 8)

{md_table(ex0["frame"], fmt="{}")}

Purity credits each cluster with its majority class, so it is
{ex0["scores"]["purity"]:.3f}. Max-matching accuracy may use each class only once, so
the three surplus clusters count as errors and accuracy drops to {ex0["acc"]:.3f}. The
Hungarian mapping (cluster → class) is `{ex0["mapping"]}`.



## 2. Experiment 1: varying the number of clusters

K-means with k = 1…15 on 5-class blobs (balanced: 5 × 240 points; imbalanced:
{DATASETS["imbalanced"]}).

{md_table(best_k)}



The one-sided metrics are pulled towards the extremes. Inverse purity and pair recall are
maximised by a single cluster (k = 1). On the imbalanced data, purity peaks at
k = {", ".join(map(str, best_k_imb["purity"]))} and pair precision at
k = {", ".join(map(str, best_k_imb["pair_precision"]))}. Both can be inflated simply by
asking for more clusters.

On the imbalanced data, every two-sided metric prefers
```

```
k = {", ".join(map(str, best_k_imb["accuracy"]))} to the true k = {TRUE_K}. The contingency
matrices show why. At k = {TRUE_K}, k-means spends a centroid splitting the 700-point class
in half, and the 50-point central class still gets no cluster of its own:

{md_table(ex0["frame_k5"], fmt="{}")}

At k = {TRUE_K - 1}, the majority class stays intact and only the tiny class is lost:

{md_table(ex0["frame_k4"], fmt="{}")}

Purity barely notices the split ({k1.at[TRUE_K, "purity"]:.3f} at k = {TRUE_K} vs
{k1.at[TRUE_K - 1, "purity"]:.3f} at k = {TRUE_K - 1}). Max-matching accuracy collapses
from {k1.at[TRUE_K - 1, "accuracy"]:.3f} to {k1.at[TRUE_K, "accuracy"]:.3f} because one
half of the majority class can no longer be matched. This is the same data and the same
pair of clusterings, but purity and accuracy reach opposite conclusions.

## 3. Experiment 2: fragmentation vs merging

Ground truth of 8 classes × 100 points; either split every class into *m* clusters or
merge *g* classes into one.

![fragmentation and merging](figures/exp2_frag_merge.png)

{md_table(fmt_scores(frag.set_index("pieces")))}

*Fragmentation (rows = pieces per class).*

{md_table(fmt_scores(merge.set_index("merged")))}

*Merging (rows = number of classes merged).*

Fragmentation is invisible to purity, pair precision and BCubed precision, all of which
stay at 1.0. Merging is invisible to inverse purity, pair recall and BCubed recall. So any
single one-sided score can rank a badly fragmented solution above a mildly merged one, or
the reverse.

### One split vs one merge

{md_table(fmt_scores(single))}

The same structural mistake costs very different amounts depending on which class it
hits. On imbalanced data, splitting the majority class in half costs pair F1 a lot because
it destroys a quadratic number of pairs. NMI, by contrast, stays relatively high
({single.at["imbalanced: split majority in 2", "nmi"]:.3f} against an ARI of
{single.at["imbalanced: split majority in 2", "ari"]:.3f}). Merging the two smallest classes
barely registers under pair-based metrics (pair F1
{single.at["imbalanced: merge two smallest", "pair_f1"]:.3f}), but max-matching accuracy
charges for every point of the unmatched class
({single.at["imbalanced: merge two smallest", "accuracy"]:.3f}).

## 4. Experiment 3: class imbalance

Four classes and 1000 points; the majority class holds a fraction *p*, and the rest is
split evenly over three minority classes.

![imbalance](figures/exp3_imbalance.png)

**When does a degenerate one-cluster answer beat a near-perfect answer that only splits
the majority class in two?**

{md_table(cross, fmt="{}")}

**Best solution per metric as imbalance grows:**

{md_table(imb_winners, fmt="{}")}

At p = 0.95 the single cluster scores purity {p95.at["single cluster", "purity"]:.3f},
accuracy {p95.at["single cluster", "accuracy"]:.3f} and pair F1
{p95.at["single cluster", "pair_f1"]:.3f}, but ARI {p95.at["single cluster", "ari"]:.3f}
and NMI {p95.at["single cluster", "nmi"]:.3f}. Metrics without chance correction inherit
the majority-class baseline, so their absolute values become uninterpretable under
imbalance. Their rankings can also flip in favour of trivial solutions.

## 5. Experiment 4: do metrics rank the same solutions the same way?

Each dataset gets 16 candidate solutions. {n_algo} are produced by algorithms: k-means for
several k, agglomerative clustering (ward, average and single linkage) and a GMM. The other
5 are controlled corruptions of the truth (10% noise, split largest class, merge two
```

```
smallest classes) plus the two degenerate answers (one cluster, all singletons). Rankings
are analysed for the full pool and for the {n_algo} algorithmic solutions alone, which is
the realistic model-selection setting.

### Full pool (16 solutions)

![ranks, full pool](figures/exp4_ranks_all.png)

{full_pool_lead}

{md_table(imb_res["all"]["winners"], fmt="{}")}

*(imbalanced data; ties share rank 1)*

Even here, the one-sided metrics crown a degenerate answer. Purity and pair precision
give their maximum of 1.0 to "all singletons" as well as to the fragmented truth, while
inverse purity and pair recall give theirs to "single cluster" as well as to the merged
truth.

### Algorithmic solutions only ({n_algo} solutions)

![ranks, algorithms](figures/exp4_ranks_algorithms.png)

![kendall, algorithms](figures/exp4_kendall_algorithms.png)

{md_table(summary)}

*"Rank spread" is the average, over solutions, of (worst rank − best rank) that the six
two-sided metrics give the same solution.*

On the imbalanced data, the best algorithm according to the two-sided metrics is
{best_algo}. They do not agree on the worst one: {worst_algo}. The disagreements are not
limited to near-ties either. The widest gap among the two-sided metrics is for
*{widest}*, which is ranked {int(algo_ranks.at[widest, widest_lo])} of {n_algo} by
{label(widest_lo)} but {int(algo_ranks.at[widest, widest_hi])} by {label(widest_hi)}.
Including one-sided metrics widens it further: over-split k-means with k = 10 is
ranked {rank_of("k-means k=10", "purity")} by purity and
{rank_of("k-means k=10", "accuracy")} by accuracy.

### Concrete reversals (imbalanced data, algorithmic solutions)

{chr(10).join(conflict_blocks)}

### Full scores (imbalanced data)

{md_table(fmt_scores(imb_res["scores"]))}

## 6. Answer: how metrics come to conflicting conclusions

1. **They measure different error types.** Purity, pair precision and BCubed precision
   penalise only merging, and their mirror images penalise only fragmentation. Any
   one-sided score can be maximised by a degenerate clustering (all singletons or a single
   cluster), so it orders solutions by their number of clusters as much as by their
   quality.
2. **They count different units.** Point-based metrics (purity, accuracy, BCubed) weight a
   class linearly by its size. Pair-based metrics (pair P/R/F, ARI) weight it
   quadratically. Errors inside the majority class therefore dominate pair metrics, and
   errors in small classes almost vanish from them.
3. **They match clusters to classes differently.** Purity lets several clusters claim the
   same class, the class F-measure lets several classes claim the same cluster, and
   max-matching accuracy allows only a one-to-one mapping. A fragmented solution is free
   under purity but pays in full under accuracy. For merges the outcome depends on the
   class sizes involved. Merging two equal classes is penalised less by the class
   F-measure ({merge_bal["class_f"]:.3f}) than by accuracy ({merge_bal["accuracy"]:.3f}),
   because one cluster can be the best match for both classes. Merging the majority with
   the smallest class reverses this (class F {merge_mix["class_f"]:.3f}, accuracy
   {merge_mix["accuracy"]:.3f}): the small class loses almost all of its precision, while
   accuracy still credits every majority point.
4. **Some correct for chance and some do not.** With a 95% majority class, putting
   everything in one cluster scores accuracy {p95_single["accuracy"]:.3f} and pair F1
   {p95_single["pair_f1"]:.3f}, while ARI = {p95_single["ari"]:.3f}. The near-perfect
   "split majority in 2" solution scores accuracy {p95_split["accuracy"]:.3f} and pair F1
   {p95_split["pair_f1"]:.3f}, but ARI {p95_split["ari"]:.3f}. Uncorrected metrics
   inherit the majority-class baseline, so they rank the useless answer higher once
   imbalance passes a metric-specific threshold. That threshold is
   {cross.iloc[:, 0][label("accuracy")]} for accuracy and
   {cross.iloc[:, 0][label("pair_f1")]} for pair F1 (see Experiment 3). The same
   comparison on the moderately imbalanced blobs (58% majority) does not flip: the single
```

```
   cluster's accuracy of {single_cluster["accuracy"]:.3f} is still below
   {split_largest["accuracy"]:.3f}.
5. **They are biased by the number of clusters.** Purity tends to rise with k (on the
   imbalanced data it peaks at k = {", ".join(map(str, best_k_imb["purity"]))}). NMI is
   lenient towards over-splitting and harsh on under-splitting. It ranks k-means k = 2
   {rank_of("k-means k=2", "nmi")} of {n_algo}, where accuracy ranks it
   {rank_of("k-means k=2", "accuracy")}, and it ranks k = 10
   {rank_of("k-means k=10", "nmi")}, where accuracy ranks it
   {rank_of("k-means k=10", "accuracy")}. Accuracy punishes any surplus cluster in full.
6. **Imbalance amplifies all of the above.** Among the algorithmic solutions, the mean
   Kendall tau between the six two-sided metrics falls from {s_bal[tau_key]:.2f} on
   balanced data to {s_imb[tau_key]:.2f} on imbalanced data. A typical pair of these
   metrics orders {100 * s_bal[dis_key]:.0f}% of solution pairs oppositely on balanced data
   and {100 * s_imb[dis_key]:.0f}% on imbalanced data, and the average rank spread
   of a solution grows from {s_bal[spread_key]:.1f} to {s_imb[spread_key]:.1f} places.

**Practical consequence:** a single number cannot certify a clustering as better. Report
a complementary pair that exposes both error types, such as pair or BCubed precision and
recall, or purity and inverse purity. Add a chance-corrected score (ARI) and show the
contingency matrix itself, which is the only view that records exactly which classes were
split or merged.

## References

- Amigó, E., Gonzalo, J., Artiles, J. & Verdejo, F. (2009). A comparison of extrinsic
  clustering evaluation metrics based on formal constraints. *Information Retrieval*,
  12(4), 461–486.
- Bagga, A. & Baldwin, B. (1998). Entity-based cross-document coreferencing using the
  vector space model. *Proc. COLING-ACL*, 79–85.
- Hubert, L. & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2,
  193–218.
- Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research
  Logistics Quarterly*, 2, 83–97.
- Larsen, B. & Aone, C. (1999). Fast and effective text mining using linear-time document
  clustering. *Proc. ACM SIGKDD*, 16–22.
- Manning, C. D., Raghavan, P. & Schütze, H. (2008). *Introduction to Information
  Retrieval*, ch. 16. Cambridge University Press.
- Strehl, A. & Ghosh, J. (2002). Cluster ensembles — a knowledge reuse framework for
  combining multiple partitions. *Journal of Machine Learning Research*, 3, 583–617.
- Vinh, N. X., Epps, J. & Bailey, J. (2010). Information theoretic measures for clusterings
  comparison. *Journal of Machine Learning Research*, 11, 2837–2854.
"""
    (OUT / "report.md").write_text(report, encoding="utf-8")


def main():
    FIG.mkdir(parents=True, exist_ok=True)
    data = {name: make_dataset(sizes) for name, sizes in DATASETS.items()}
    ex0 = exp_datasets(data)
    e1 = exp_vary_k(data)
    e2 = exp_frag_merge()
    e3 = exp_imbalance()
    e4 = exp_rankings(data)
    write_report(ex0, e1, e2, e3, e4)
    print(e1[1].to_string())
    print(e3[1].to_string())
    print(e4[1].to_string())
    print(e4[0]["imbalanced"]["algorithms"]["winners"].to_string())
    print(f"Report written to {OUT / 'report.md'}")


if __name__ == "__main__":
    main()
```

experiments/build_pdf.py

```
"""Build the submission PDF from results/report.md plus a source-code appendix.

Usage:  python experiments/build_pdf.py   (run experiments/run_all.py first)

The HTML is printed to PDF with headless Microsoft Edge or Google Chrome.
"""

from __future__ import annotations

import datetime as dt
```

```python
import html
import io
import os
import shutil
import subprocess
import sys
import unittest
from pathlib import Path

import markdown
from pygments import highlight
from pygments.formatters import HtmlFormatter
from pygments.lexers import PythonLexer

ROOT = Path(__file__).resolve().parents[1]
RESULTS = ROOT / "results"
HTML_OUT = RESULTS / "report.html"
PDF_OUT = ROOT / "Clustering_Evaluation_Metrics_Report.pdf"

TITLE = "When Clustering Metrics Disagree"
SUBTITLE = (
    "A contingency-matrix study of purity, maximum-matching accuracy, precision, recall "
    "and F-measure under fragmentation, merging and class imbalance"
)
FIELDS = ["Name", "Roll / Student ID", "Course", "Instructor", "Date of submission"]
SOURCE_FILES = [
    "clustermetrics/__init__.py",
    "clustermetrics/contingency.py",
    "clustermetrics/perturb.py",
    "clustermetrics/ranking.py",
    "experiments/run_all.py",
    "experiments/build_pdf.py",
    "tests/test_metrics.py",
]
BROWSERS = [
    r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe",
    r"C:\Program Files\Microsoft\Edge\Application\msedge.exe",
    r"C:\Program Files\Google\Chrome\Application\chrome.exe",
    "msedge",
    "google-chrome",
    "chromium",
]

CSS = """
@page { size: A4; margin: 16mm 15mm 16mm 15mm; }
* { -webkit-print-color-adjust: exact; print-color-adjust: exact; }
body { font-family: "Segoe UI", Calibri, Arial, sans-serif; font-size: 10.2pt;
       line-height: 1.45; color: #1d1d1f; }
h1, h2, h3 { color: #1f3a5f; line-height: 1.25; }
h2 { font-size: 16pt; border-bottom: 2px solid #1f3a5f; padding-bottom: 3px;
     break-before: page; margin-top: 0; }
h3 { font-size: 12pt; margin-top: 18px; break-after: avoid; }
p, li { text-align: justify; }
code { font-family: Consolas, "Courier New", monospace; font-size: 8.8pt;
       background: #f2f4f7; padding: 0 2px; border-radius: 2px; }
table { border-collapse: collapse; margin: 8px 0 12px; font-size: 7.8pt; width: 100%; }
th, td { border: 1px solid #c7cdd6; padding: 2.5px 5px; text-align: left; vertical-align: top; }
th { background: #e6ecf4; }
tr { break-inside: avoid; }
tr:nth-child(even) td { background: #f8f9fb; }
img { max-width: 100%; display: block; margin: 10px auto; break-inside: avoid; }
em { color: #333; }

.title-page { height: 255mm; display: flex; flex-direction: column; justify-content: center;
              text-align: center; break-after: page; }
.title-page h1 { font-size: 28pt; margin-bottom: 8px; border: none; }
.title-page .subtitle { font-size: 13pt; color: #444; max-width: 150mm; margin: 0 auto 26px;
                        text-align: center; }
.title-page .question { font-size: 11.5pt; margin: 0 auto 50px; max-width: 150mm;
                        padding: 10px 14px; border-left: 4px solid #1f3a5f;
                        background: #f2f5fa; text-align: left; }
.fields { width: 125mm; margin: 0 auto; border: none; font-size: 11.5pt; }
.fields td { border: none; background: none !important; padding: 9px 4px; }
.fields td.label { width: 45mm; font-weight: 600; color: #1f3a5f; }
.fields td.blank { border-bottom: 1px solid #555; }

h2#introduction-and-method, h2#references { break-before: auto; margin-top: 26px; }

```

```
.toc-page { break-after: page; }
.toc-page h2 { break-before: auto; }
.toc ul { list-style: none; padding-left: 14px; }
.toc > ul { padding-left: 0; }
.toc-page li { text-align: left; margin: 3px 0; }
.toc-page a { color: #1d1d1f; text-decoration: none; }

.appendix h3 { font-family: Consolas, monospace; font-size: 10.5pt; background: #1f3a5f;
               color: white; padding: 4px 8px; margin: 18px 0 0; break-after: avoid; }
.appendix .highlight pre { font-size: 7.3pt; line-height: 1.32; margin: 0 0 10px;
               padding: 6px 8px; border: 1px solid #d0d5dd; background: #fbfbfc;
               white-space: pre-wrap; word-break: break-word; }
.appendix .highlight .linenos { color: #9aa3ad; padding-right: 6px; }
pre.output { font-size: 8pt; background: #f2f4f7; padding: 8px; border: 1px solid #d0d5dd;
             white-space: pre-wrap; }
"""


def title_page() -> str:
    rows = "\n".join(
        f'<tr><td class="label">{f}</td><td class="blank"> </td></tr>' for f in FIELDS
    )
    return f"""
<section class="title-page">
  <h1>{TITLE}</h1>
  <p class="subtitle">{SUBTITLE}</p>
  <p class="question"><strong>Research question.</strong> How can evaluation metrics produce
  conflicting conclusions about clustering quality?</p>
  <table class="fields">{rows}</table>
</section>"""


def report_body() -> tuple[str, str]:
    text = (RESULTS / "report.md").read_text(encoding="utf-8")
    lines = text.splitlines()
    # The title page already carries the H1 title and the research question.
    body_start = next(i for i, ln in enumerate(lines) if ln.startswith("## "))
    body = "\n".join(lines[body_start:])
    md = markdown.Markdown(
        extensions=["tables", "fenced_code", "toc", "attr_list"],
        extension_configs={"toc": {"toc_depth": "2-3"}},
    )
    return md.convert(body), md.toc


def run_tests() -> str:
    stream = io.StringIO()
    suite = unittest.defaultTestLoader.discover(str(ROOT / "tests"), top_level_dir=str(ROOT))
    result = unittest.TextTestRunner(stream=stream, verbosity=2).run(suite)
    status = "OK" if result.wasSuccessful() else "FAILED"
    return f"{stream.getvalue().strip()}\n\nStatus: {status}"


def appendix() -> str:
    formatter = HtmlFormatter(linenos="inline", style="friendly")
    parts = [
        '<section class="appendix">',
        "<h2 id='appendix-a'>Appendix A. Reproducing the results</h2>",
        "<p>All numbers, tables and figures in this document are produced by the code in "
        "Appendix B. From the project root:</p>",
        "<pre class='output'>pip install -r requirements.txt\n"
        "python -m unittest discover -s tests -t .\n"
        "python experiments/run_all.py\n"
        "python experiments/build_pdf.py</pre>",
        "<p>The unit tests check every metric against hand-computed textbook values "
        "(Manning, Raghavan &amp;amp; Sch&amp;uuml;tze, Fig. 16.4) and against scikit-learn's "
        "ARI, NMI, Fowlkes&amp;ndash;Mallows and Rand index. Output of the test run used for "
        "this document:</p>",
        f"<pre class='output'>{html.escape(run_tests())}</pre>",
        "<h2 id='appendix-b'>Appendix B. Source code</h2>",
    ]
    for rel in SOURCE_FILES:
        code = (ROOT / rel).read_text(encoding="utf-8")
        parts.append(f"<h3>{rel}</h3>")
        parts.append(highlight(code, PythonLexer(), formatter))
    parts.append("</section>")
    return "\n".join(parts), formatter.get_style_defs(".highlight")

```

```

def build_html() -> Path:
    body, toc = report_body()
    appendix_html, pygments_css = appendix()
    toc_html = (
        '<section class="toc-page"><h2>Contents</h2>'
        f'<div class="toc">{toc}</div>'
        "<ul style='list-style:none;padding-left:0'>"
        "<li><a href='#appendix-a'>Appendix A. Reproducing the results</a></li>"
        "<li><a href='#appendix-b'>Appendix B. Source code</a></li></ul>"
        "</section>"
    )
    doc = f"""<!DOCTYPE html>
<html lang="en"><head><meta charset="utf-8"><title>{TITLE}</title>
<style>{CSS}\n{pygments_css}</style></head>
<body>
{title_page()}
{toc_html}
{body}
{appendix_html}
<p style="margin-top:20px;color:#888;font-size:8pt">Generated {dt.date.today():%d %B %Y}.</p>
</body></html>"""
    HTML_OUT.write_text(doc, encoding="utf-8")
    return HTML_OUT


def find_browser() -> str:
    for candidate in BROWSERS:
        path = candidate if os.path.isabs(candidate) else shutil.which(candidate)
        if path and Path(path).exists():
            return path
    raise SystemExit("No Edge/Chrome found; open results/report.html and print it to PDF.")


def print_pdf(html_path: Path, pdf_path: Path) -> None:
    if pdf_path.exists():
        pdf_path.unlink()
    subprocess.run(
        [
            find_browser(),
            "--headless=new",
            "--disable-gpu",
            "--no-pdf-header-footer",
            "--virtual-time-budget=15000",
            f"--print-to-pdf={pdf_path}",
            html_path.as_uri(),
        ],
        check=True,
        capture_output=True,
        timeout=180,
    )
    if not pdf_path.exists():
        raise SystemExit("Browser exited without writing the PDF.")


def main():
    if not (RESULTS / "report.md").exists():
        raise SystemExit("results/report.md missing; run experiments/run_all.py first.")
    html_path = build_html()
    print_pdf(html_path, PDF_OUT)
    print(f"PDF written to {PDF_OUT} ({PDF_OUT.stat().st_size / 1024:.0f} KB)")


if __name__ == "__main__":
    sys.exit(main())
```

tests/test_metrics.py

```

import unittest

import numpy as np
from sklearn import metrics as skm

from clustermetrics import (
    adjusted_rand_index,
    bcubed,
```

```
    class_f_measure,
    contingency_matrix,
    evaluate,
    inverse_purity,
    max_matching_accuracy,
    normalized_mutual_info,
    pair_counts,
    pair_precision_recall_f,
    purity,
)
from clustermetrics import perturb


# Manning, Raghavan & Schuetze (IR book, Fig. 16.4): 17 points, 3 clusters.
# Classes: x=0, o=1, d=2.
IR_TRUE = np.array([0] * 5 + [1] * 1 + [0] * 1 + [1] * 4 + [2] * 1 + [0] * 2 + [2] * 3)
IR_PRED = np.array([0] * 6 + [1] * 6 + [2] * 5)


class TestContingency(unittest.TestCase):
    def test_matrix_matches_sklearn(self):
        rng = np.random.default_rng(0)
        y, p = rng.integers(0, 4, 300), rng.integers(0, 6, 300)
        np.testing.assert_array_equal(
            contingency_matrix(y, p).matrix, skm.cluster.contingency_matrix(y, p)
        )

    def test_arbitrary_label_values(self):
        c = contingency_matrix(["a", "a", "b"], [10, 20, 20])
        np.testing.assert_array_equal(c.matrix, [[1, 1], [0, 1]])
        self.assertEqual(list(c.classes), ["a", "b"])


class TestTextbookValues(unittest.TestCase):
    def test_purity(self):
        self.assertAlmostEqual(purity(IR_TRUE, IR_PRED), 12 / 17)

    def test_pair_counts(self):
        pc = pair_counts(IR_TRUE, IR_PRED)
        self.assertEqual((pc["tp"], pc["fp"], pc["fn"], pc["tn"]), (20, 20, 24, 72))

    def test_pair_prf(self):
        p, r, f = pair_precision_recall_f(IR_TRUE, IR_PRED)
        self.assertAlmostEqual(p, 0.5)
        self.assertAlmostEqual(r, 20 / 44)
        self.assertAlmostEqual(f, 2 * 0.5 * (20 / 44) / (0.5 + 20 / 44))


class TestAgainstSklearn(unittest.TestCase):
    def setUp(self):
        rng = np.random.default_rng(42)
        self.cases = [
            (rng.integers(0, k1, n), rng.integers(0, k2, n))
            for n, k1, k2 in [(50, 2, 2), (200, 3, 7), (500, 6, 3), (120, 5, 5)]
        ]
        self.cases.append((IR_TRUE, IR_PRED))

    def test_ari(self):
        for y, p in self.cases:
            self.assertAlmostEqual(adjusted_rand_index(y, p), skm.adjusted_rand_score(y, p))

    def test_nmi(self):
        for y, p in self.cases:
            self.assertAlmostEqual(
                normalized_mutual_info(y, p), skm.normalized_mutual_info_score(y, p)
            )

    def test_pair_precision_recall_via_fowlkes_mallows(self):
        for y, p in self.cases:
            prec, rec, _ = pair_precision_recall_f(y, p)
            self.assertAlmostEqual(np.sqrt(prec * rec), skm.fowlkes_mallows_score(y, p))

    def test_rand_index(self):
        for y, p in self.cases:
            pc = pair_counts(y, p)
            ri = (pc["tp"] + pc["tn"]) / sum(pc.values())
            self.assertAlmostEqual(ri, skm.rand_score(y, p))

```

```

class TestProperties(unittest.TestCase):
    def setUp(self):
        self.y = np.repeat(np.arange(4), [40, 30, 20, 10])

    def test_perfect_labelling(self):
        perm = np.array([3, 0, 2, 1])[self.y]
        for name, value in evaluate(self.y, perm).items():
            if name != "n_clusters":
                self.assertAlmostEqual(value, 1.0, msg=name)

    def test_accuracy_matches_permutation(self):
        p = perturb.label_noise(self.y, 0.2, rng=1)
        acc, mapping = max_matching_accuracy(self.y, p)
        mapped = np.array([mapping[v] for v in p])
        self.assertAlmostEqual(acc, (mapped == self.y).mean())

    def test_accuracy_penalises_extra_clusters(self):
        p = perturb.fragment(self.y, [0], 2, rng=0)
        acc, _ = max_matching_accuracy(self.y, p)
        self.assertAlmostEqual(acc, 80 / 100)

    def test_fragmentation_keeps_purity_and_precision_perfect(self):
        p = perturb.fragment_all(self.y, 3, rng=0)
        self.assertEqual(purity(self.y, p), 1.0)
        self.assertEqual(pair_precision_recall_f(self.y, p)[0], 1.0)
        self.assertEqual(bcubed(self.y, p)[0], 1.0)
        self.assertLess(inverse_purity(self.y, p), 1.0)

    def test_merging_keeps_inverse_purity_and_recall_perfect(self):
        p = perturb.merge(self.y, [[0, 1]])
        self.assertEqual(inverse_purity(self.y, p), 1.0)
        self.assertEqual(pair_precision_recall_f(self.y, p)[1], 1.0)
        self.assertEqual(bcubed(self.y, p)[1], 1.0)
        self.assertLess(purity(self.y, p), 1.0)

    def test_perturbations_use_original_class_ids(self):
        y = np.array([10, 10, 10, 10, 20, 20, 30, 30])
        self.assertEqual(np.unique(perturb.fragment(y, [10], 2, rng=0)).size, 4)
        self.assertEqual(np.unique(perturb.merge(y, [[20, 30]])).size, 2)
        with self.assertRaises(ValueError):
            perturb.merge(y, [[1, 2]])
        with self.assertRaises(ValueError):
            perturb.fragment(y, [20], 3)

    def test_degenerate_solutions(self):
        one = evaluate(self.y, perturb.single_cluster(self.y))
        self.assertAlmostEqual(one["purity"], 0.4)
        self.assertAlmostEqual(one["accuracy"], 0.4)
        self.assertAlmostEqual(one["pair_recall"], 1.0)
        self.assertAlmostEqual(one["ari"], 0.0)
        self.assertAlmostEqual(one["nmi"], 0.0)
        sing = evaluate(self.y, perturb.singletons(self.y))
        self.assertAlmostEqual(sing["purity"], 1.0)
        self.assertAlmostEqual(sing["pair_precision"], 1.0)
        self.assertAlmostEqual(sing["pair_recall"], 0.0)
        self.assertAlmostEqual(sing["ari"], 0.0)

    def test_class_f_bounds(self):
        rng = np.random.default_rng(3)
        for _ in range(20):
            p = rng.integers(0, rng.integers(1, 8), self.y.size)
            self.assertTrue(0.0 <= class_f_measure(self.y, p) <= 1.0)


if __name__ == "__main__":
    unittest.main()
```